

INTERVIEW PREPARATION HANDBOOK

Constraints

for Synthesis and STA

CHAPTER 1

SDC Fundamentals and Synthesis Basics

80 Interview Questions • 4 Difficulty Levels • Concept Primer, Worked Examples & Diagrams • For ASIC / Physical Design Engineers

Table of Contents

Chapter 1: SDC Fundamentals & Synthesis Basics

Chapter Overview

Concept Primer

Level 1 — Basic (0-2 Years)

Level 2 — Intermediate (2-5 Years)

Level 3 — Advanced (5-8 Years)

Level 4 — Expert (8+ Years: Senior/Lead/Staff)

Chapter 1 — Chapter Summary

Top 20 Quick Revision Questions (Chapter 1)

Important Definitions (Chapter 1)

Important Commands (Chapter 1)

Quick Revision Notes (Chapter 1)

Chapter 1: SDC Fundamentals & Synthesis Basics

Chapter Overview

This chapter establishes what SDC (Synopsys Design Constraints) is, why it exists, and how it drives both synthesis and static timing analysis. Every later chapter — clocks, generated clocks, I/O delays, exceptions — is a specific category of SDC constraint; this chapter is the foundation that explains what constraints *are*, the timing-path model they describe, the four categories they fall into, and the Tcl substrate they are written in. It also covers synthesis basics, because SDC is first consumed by synthesis, where the same timing intent that later governs signoff first shapes the gate-level netlist.

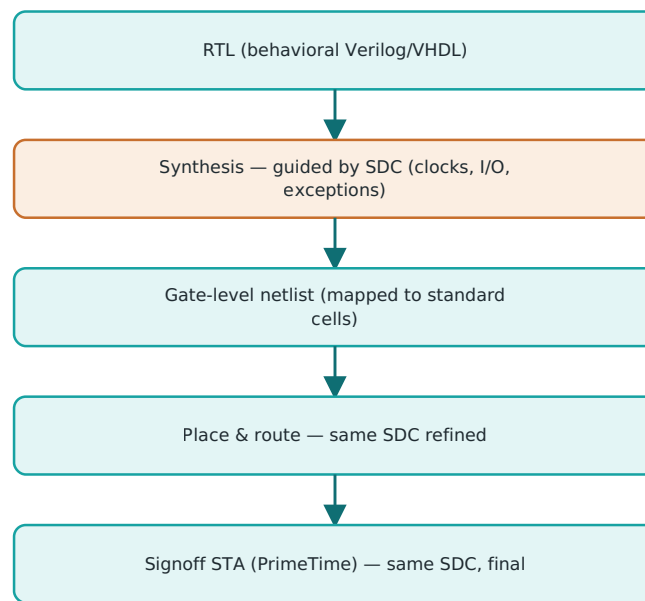
Why it matters in interviews: constraints are the single largest source of real timing error — not because tools are wrong, but because the constraints told them the wrong thing. Interviewers use SDC fundamentals to test whether you understand that a timing tool does *exactly* what the constraints say, that the dangerous errors are silent (a missing or mis-matched constraint leaves paths unchecked or wrongly checked with no error), and that the same SDC must mean the same thing across synthesis, place-and-route, and signoff.

Where it is used in real projects: SDC is written once (ideally from a single source of intent) and consumed at every stage — synthesis to shape the netlist, P&R to guide optimization, and signoff to produce the authoritative timing verdict. Getting the fundamentals right — correct clocks, correct I/O, correct exceptions, consistent across stages and tools — is what makes the whole timing flow trustworthy.

Concept Primer

1.1 What SDC is and why it exists

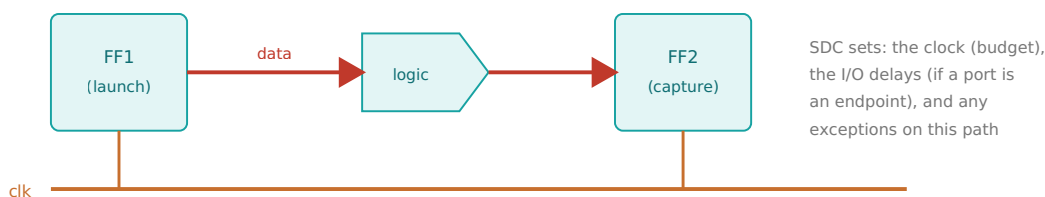
SDC (Synopsys Design Constraints) is a standard, text-based format for describing a design's *timing intent* and *environment* to synthesis and timing tools. A netlist describes what the logic *is*; SDC describes how it must *behave in time* — what the clocks are, when signals arrive at and must leave the ports, and which timing relationships are real. Without SDC a timing tool has no notion of frequency, no boundary timing, and no way to know which structural paths are real functional paths. SDC supplies exactly that missing context, and it does so in a tool-neutral, portable form so the *same* intent can flow through the whole implementation flow.

SDC constrains synthesis: the same intent flows from RTL to signoff

The essential idea is that one description of timing intent guides every stage. Synthesis reads the SDC to build a netlist that can meet timing; place-and-route reads it (refined) to optimize placement and routing; signoff STA reads it (final) to produce the authoritative verdict. When the SDC means the same thing at every stage, the stages agree; when it drifts (a constraint present at signoff but not at synthesis, or interpreted differently), the design can pass one stage and fail another. So SDC is not just a signoff input — it is the shared contract that keeps synthesis, P&R, and STA aligned.

1.2 The timing path — the unit SDC constrains

Everything SDC does is ultimately about **timing paths**. A timing path runs from a start point (a launch flip-flop's clock pin, or an input port) through combinational logic to an end point (a capture flip-flop's data pin, or an output port). STA computes the delay along each path and checks it against the time available; SDC supplies the information needed to define that "time available."

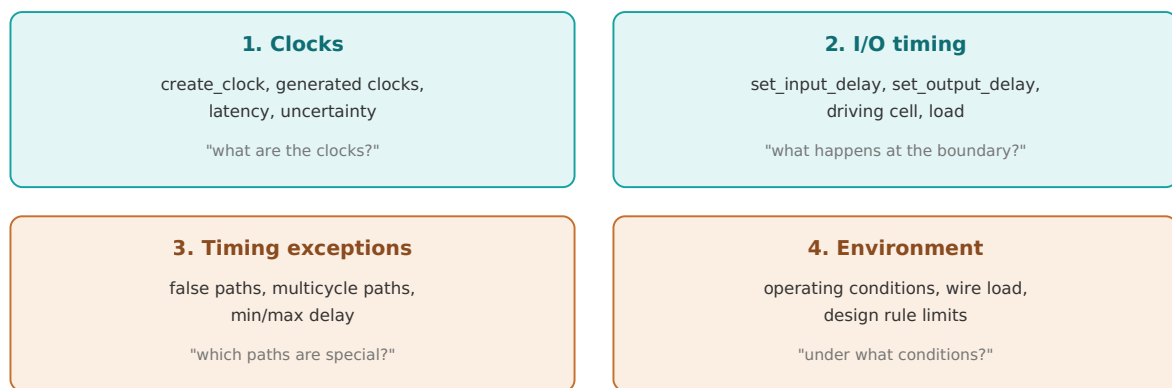
The unit SDC constrains: a timing path (launch flop to capture flop)

For a path between two flops, the clock definition sets the budget (one period for a single-cycle path). For a path that starts at an input port or ends at an output port, an I/O delay constraint supplies the external timing context. And any path may carry an *exception* (false, multicycle, or an explicit min/max delay) that changes how — or whether — it is checked. So the four constraint categories all exist to correctly characterize timing paths: what clocks them, what happens at their boundaries, and which ones are special.

1.3 The four categories of constraints

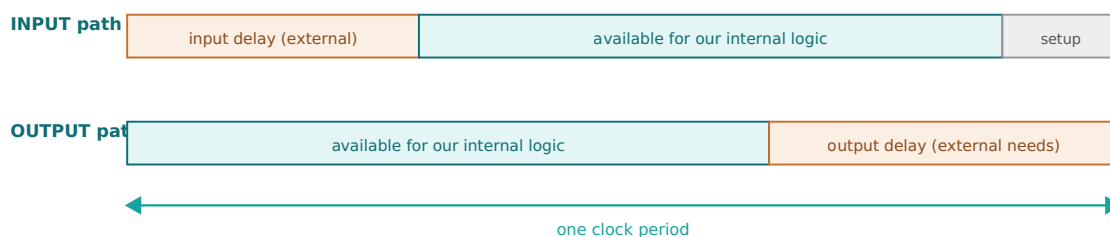
Almost every SDC command falls into one of four categories, each answering a different question the timing tool needs answered:

The four things SDC must tell the tool



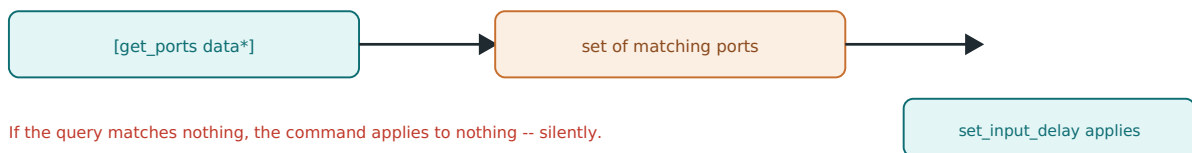
Clocks answer "what are the clocks?" — their waveforms, relationships, latency, and uncertainty (the entire previous chapter). **I/O timing** answers "what happens at the boundary?" — when data arrives at inputs and must leave outputs, and the external drive/load. **Timing exceptions** answer "which paths are special?" — false paths (don't time), multicycle paths (more than one cycle allowed), and explicit min/max delays. **Environment** answers "under what conditions?" — operating conditions (PVT), and design-rule limits (max transition, max capacitance). Mastering SDC is really mastering these four categories and how they interact: the clocks set budgets, the I/O delays extend those budgets to the boundary, the exceptions modify specific paths, and the environment sets the conditions and rules everything is checked under.

I/O delays split the period between external and internal timing



Timing exceptions override the default single-cycle check**1.4 SDC is Tcl**

A constraint = a command applied to queried objects



SDC is not a static data format — it is **Tcl** (Tool Command Language). Each constraint is a Tcl command (`create_clock`, `set_input_delay`, `set_false_path`, ...) that operates on design objects returned by *object-query* commands (`get_ports`, `get_pins`, `get_clocks`, `get_cells`). This has important consequences.

SDC is Tcl: commands + object queries, evaluated in order



Because it is Tcl, constraints are evaluated top-to-bottom: order matters, variables and loops are allowed, and a query that matches nothing silently constrains nothing (a common bug).

Danger: `get_ports clk_*` matching zero objects -> the constraint is silently skipped.

Because SDC is Tcl, it is evaluated *top to bottom* — **order matters** (a later constraint can override an earlier one; an object must exist when referenced). Variables, loops, and procedures are allowed, which enables powerful, parameterized constraint generation. But it also introduces a characteristic danger: a query that matches *nothing* (a typo'd port name, a wrong hierarchy path) silently constrains nothing — the command runs without error but applies to zero objects, leaving the intended paths unconstrained. This is why disciplined constraint writing checks that queries actually match, and why "the SDC ran without error" is never proof the constraints are correct.

1.5 Synthesis basics: where SDC is first used

Synthesis translates RTL into a gate-level netlist mapped to a standard-cell library, and it is *timing-driven* — it uses the SDC to decide how hard to work on each path. The clock period tells synthesis the setup budget, so it knows how much logic can fit between flops and where to insert pipelining or restructure logic; the I/O delays tell it the boundary budgets; the exceptions tell it which paths not to over-optimize (a false path shouldn't consume effort). If the SDC is wrong at synthesis — say, a missing clock or a too-loose period — synthesis builds a netlist that is under- or over-optimized, and the problem propagates downstream. Synthesis also performs logic optimization (mapping, sizing, buffering) guided by the constraints and the library's timing. The key point is that SDC's influence begins at synthesis, not at signoff: the same clocks, I/O delays, and exceptions first shape the netlist, so they must be correct — and consistent with what signoff will later use — from the very start.

1.6 Why constraints are where designs go wrong

The danger: constraint errors are SILENT (no violation appears)

missing clock -> flops unlocked -> paths NOT timed (report looks clean)

wrong / over-broad exception -> real paths excepted -> violation HIDDEN

query matches nothing -> constraint applied to ZERO objects (runs cleanly)

None produce an error. A clean report is necessary but NOT sufficient -> verify with `check_timing`.

The recurring theme of the whole handbook begins here: the timing tool does *exactly* what the constraints say, no more and no less. A missing clock leaves paths unchecked (silently). A missing I/O delay leaves boundary paths unconstrained (silently). A wrong exception hides a real violation (silently) or a missing one floods the report with false ones. An object query that matches nothing applies a constraint to nothing (silently). None of these produce an error — they produce *wrong or absent checking* that looks clean. And because the same SDC drives synthesis, P&R, and signoff, an inconsistency between stages (a constraint applied differently, or present in one flow and not another) can make a design pass one stage and fail the next. This is why constraint correctness, completeness, and consistency — verified, not assumed — are the foundation of trustworthy timing, and why the fundamentals in this chapter underpin everything that follows.

Level 1 — Basic (0-2 Years)

Q1. What is SDC and what is it used for?

Why the interviewer asks: The most basic check that you know what constraints are and why they exist.

Expected answer: SDC (Synopsys Design Constraints) is a standard, Tcl-based text format for describing a design's timing intent and environment to synthesis and timing-analysis tools. It tells the tool what it cannot learn from the netlist alone: what the clocks are (frequency and waveform), when signals arrive at and must leave the I/O ports, which timing paths are special (false or multicycle), and under what operating conditions and design rules the design must work. It's used at every stage of the flow — synthesis reads it to build a netlist that can meet timing, place-and-route reads it to guide optimization, and signoff STA reads it to produce the authoritative timing verdict. The essential point is that the netlist describes what the logic *is*, while SDC describes how it must *behave in time*; without SDC the tool has no notion of frequency or boundary timing and cannot meaningfully check timing at all.

Follow-up questions: 1. What does the netlist describe versus what SDC describes? 2. Name the stages of the flow that consume SDC.

Follow-up answers: 1. The netlist describes the logic structure (what gates exist and how they're connected — what the design *is*); SDC describes the timing intent and environment (clocks, I/O timing, exceptions, conditions — how it must *behave in time*). 2. Synthesis (to build a timing-meeting netlist), place-and-route (to guide placement/routing optimization), and signoff static timing analysis (to produce the final timing verdict) — the same SDC intent flows through all of them.

Q2. What are the main categories of constraints in an SDC file?

Why the interviewer asks: Tests whether you can organize SDC into its conceptual groups.

Expected answer: Almost every SDC command falls into one of four categories. **(1) Clock constraints** — defining the clocks: `create_clock`, generated clocks, latency, uncertainty (what the clocks are). **(2) I/O timing constraints** — the boundary timing: `set_input_delay`, `set_output_delay`, and the drive/load environment like `set_driving_cell` and `set_load` (what happens at the ports). **(3) Timing exceptions** — paths that aren't ordinary single-cycle paths: `set_false_path`, `set_multicycle_path`, `set_max_delay` / `set_min_delay` (which paths are special). **(4) Environmental/design-rule constraints** — the conditions and limits: operating conditions (PVT), `set_max_transition`, `set_max_capacitance` (under what conditions and rules). Understanding these four categories is really the structure of mastering SDC — the clocks set the budgets, the I/O delays extend them to the boundary, the exceptions modify specific paths, and the environment sets the conditions everything is checked under.

Follow-up questions: 1. Which category does `set_false_path` belong to? 2. Which category sets the operating conditions (PVT)?

Follow-up answers: 1. Timing exceptions — it tells the tool a path is not a real functional timing path and should not be checked, which is an exception to ordinary single-cycle timing. 2. The environmental/design-rule category — operating conditions specify the process, voltage, and temperature (PVT) the design is analyzed under, alongside design-rule limits like max transition and max capacitance.

Q3. Why is SDC written in Tcl, and what practical consequence does that have?

Why the interviewer asks: Tests understanding that SDC is executable, not a static data file.

Expected answer: SDC is Tcl (Tool Command Language), so each constraint is actually a Tcl command that runs, operating on design objects returned by query commands like `get_ports` or `get_clocks`. The practical consequences: **(1)** Constraints are evaluated *top to bottom* — order matters, so a later command can override an earlier one, and an object must exist when it's referenced. **(2)** You can use Tcl features — variables, loops, procedures, conditionals — to write powerful, parameterized constraints (e.g. loop over a bus, define periods from a variable). **(3)** Most importantly, a query that matches *nothing* (a typo'd name, wrong hierarchy) silently applies the constraint to zero objects — the command runs without error but does nothing, leaving paths unconstrained. This "silent no-op" is a classic SDC bug. So being Tcl makes SDC flexible and programmable, but it also means "the file ran without error" doesn't prove the constraints did what you intended — you must verify the queries actually matched.

Follow-up questions: 1. Why does order matter in an SDC file? 2. What happens if `get_ports` matches no objects?

Follow-up answers: 1. Because SDC is Tcl, evaluated top to bottom — a later constraint can override an earlier one, and any object referenced must already exist at that point; so the sequence of commands affects the final result. 2. The constraint using that query is silently applied to zero objects — the command runs without error but has no effect, leaving the intended paths unconstrained. This is a common, silent SDC bug that a clean run doesn't reveal.

Q4. What is a timing path, and what are its start and end points?

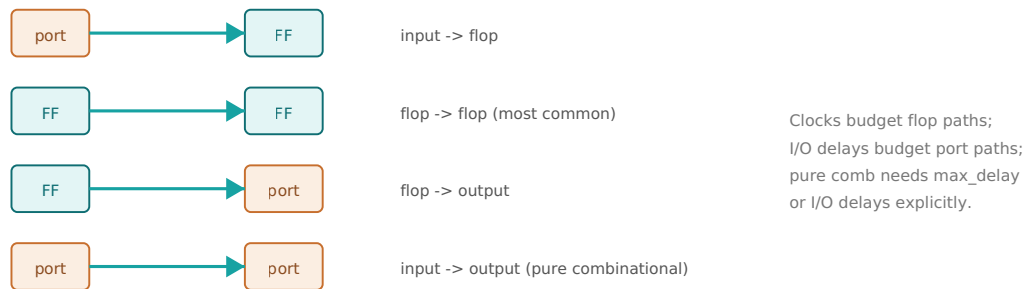
Why the interviewer asks: The timing path is the fundamental unit; everything references it.

Expected answer: A timing path is the route a signal takes from a start point, through combinational logic, to an end point, and it's the unit STA analyzes and SDC constrains.

Start points are either a clock pin of a flip-flop (where data is launched by a clock edge) or

an input port (where data enters the design). **End points** are either the data input pin of a flip-flop (where data is captured) or an output port (where data leaves). So there are four path types by their endpoints: input-port to flop, flop to flop, flop to output-port, and input-port to output-port (purely combinational through the design). The tool computes the delay along each path and checks it against the available time. SDC's job is to supply what defines that available time: the clock (for flop endpoints, giving the budget) and the I/O delays (for port endpoints, giving the external context). Understanding path types matters because each is constrained differently — flop-to-flop by clocks, port paths by I/O delays.

Four timing-path types by their endpoints



Follow-up questions: 1. What are the two kinds of start point? 2. What constrains a flop-to-flop path versus an input-port-to-flop path?

Follow-up answers: 1. A flip-flop's clock pin (data launched by a clock edge) or an input port (data entering the design from outside). 2. A flop-to-flop path is constrained by the clock definition (the period sets the budget); an input-port-to-flop path is constrained by the input delay (`set_input_delay`), which supplies the external arrival context, plus the capturing clock.

Q5. What is the difference between synthesis and static timing analysis in how they use SDC?

Why the interviewer asks: Tests understanding that SDC serves both, differently.

Expected answer: Both read the same SDC, but for different purposes. **Synthesis** uses SDC to *build and optimize* the netlist — the clock period tells it the setup budget so it knows how much logic can fit between flops, whether to pipeline or restructure, and how hard to work on each path; the I/O delays give boundary budgets; exceptions tell it which paths not to waste effort on. Synthesis is *constructive* — it creates gates guided by the constraints. **Static timing analysis** uses the same SDC to *verify* an existing netlist — it computes path delays and checks them against the budgets the SDC defines, producing slack and pass/fail. STA is *analytical* — it checks, it doesn't build. The crucial point is that they must use *consistent* SDC: if synthesis optimizes against one interpretation and signoff checks against another, the design can be built to meet a target it isn't then verified against (or vice versa). So SDC is the shared contract — synthesis builds to it, STA verifies against it, and they agree only if the constraints mean the same thing to both.

Follow-up questions: 1. Is synthesis constructive or analytical? What about STA? 2. Why must synthesis and STA use consistent SDC?

Follow-up answers: 1. Synthesis is constructive — it builds and optimizes the netlist guided by the constraints; STA is analytical — it verifies an existing netlist against the constraints, computing slack and pass/fail without changing the design. 2. Because if they use different (inconsistent) constraints, synthesis optimizes toward one target while signoff checks against another — the design could be built to meet a budget it's never verified against, or verified against a budget it wasn't built for, so the stages disagree and a design can pass one and fail the other.

Q6. What does `create_clock` do, and why is it usually the first constraint?

Why the interviewer asks: Reconnects to clocks as the foundation, from the SDC-structure view.

Expected answer: `create_clock` defines a clock — its period and waveform — at a source port or pin, establishing the periodic timing reference that launch and capture edges are derived from. It's usually the first constraint (after any setup/library commands) because virtually everything else depends on clocks existing: I/O delays are referenced to a clock, exceptions often reference clocks, and every flop-to-flop path is timed against a clock's period. If you tried to write I/O delays or time paths before defining the clocks, there'd be nothing to reference. So the natural and standard order is clocks first, then I/O timing, then exceptions — mirroring the dependency structure. Concretely, `create_clock -name CLK -period 10 [get_ports clk]` defines a 10 ns clock at the clk port. Getting the clock right is the highest-priority constraint because it's the foundation the rest build on — which is exactly why it comes first and why clock errors are the most pervasive.

Follow-up questions: 1. Why are clocks defined before I/O delays? 2. What two things does `create_clock` primarily specify?

Follow-up answers: 1. Because I/O delays (and many exceptions) are *referenced to* a clock — they specify timing relative to a clock edge — so the clock must exist first; more broadly, everything depends on clocks, so defining them first matches the dependency order. 2. The clock's period (the cycle time, setting the frequency and the setup budget) and its waveform (the positions of the rising and falling edges within the period, i.e. duty cycle and phase).

Q7. What is `set_input_delay` and what does it model?

Why the interviewer asks: Basic I/O constraint understanding.

Expected answer: `set_input_delay` models the timing of data arriving at an input port relative to a clock edge — specifically, how long *after* the reference clock edge the data becomes valid at the port. It captures the external world's contribution: the data at our

input port came from some external device that launched it on a clock edge and it traveled through external delay to reach us, so by the time it arrives at our port, some portion of the clock period is already consumed. `set_input_delay` tells the tool how much, so it correctly times the internal path from the port to the capturing flop with only the *remaining* budget. For example, `set_input_delay 3 -clock CLK [get_ports data_in]` says the data arrives 3 ns after CLK's edge, leaving (period – 3 ns – internal setup) for the internal logic. Without it, the input path is effectively unconstrained or wrongly constrained. It's how the boundary timing contract with the outside world is expressed to the tool.

Follow-up questions: 1. What does the input delay value represent physically? 2. What clock is the input delay referenced to?

Follow-up answers: 1. The external delay — the time after the reference clock edge that the data takes to become valid at our input port, reflecting the external device's clock-to-output plus the interconnect delay before our port; it's the portion of the period already consumed outside our design. 2. The clock specified with `-clock` in the command — the clock relative to whose edge the external data timing is defined (the clock the external launching device and our capturing logic share, or a virtual clock modeling it).

Q8. What is `set_output_delay` and what does it model?

Why the interviewer asks: The output-side counterpart to input delay.

Expected answer: `set_output_delay` models the timing requirement at an output port relative to a clock edge — how much time *before* the external clock edge the data must be valid at our output port. It captures what the external world *needs*: the data leaving our output port goes to some external device that will capture it on a clock edge, and that device needs the data set up before its edge (plus external interconnect delay), so our data must leave early enough. `set_output_delay` tells the tool how much of the period the external side needs, so it correctly times the internal path from the launching flop to the output port, leaving only the remaining budget for our internal logic. For example, `set_output_delay 4 -clock CLK [get_ports data_out]` says the external side needs the data 4 ns before CLK's edge, so our internal path must deliver it by then. It's the output-side half of the boundary timing contract — input delay says when data arrives, output delay says when data must leave.

Follow-up questions: 1. What does the output delay value represent physically? 2. How do input and output delays together define the boundary contract?

Follow-up answers: 1. The external requirement — the time before the external clock edge that the data must be valid at our output port, reflecting the external capturing device's setup requirement plus the interconnect delay to it; it's the portion of the period the external side needs. 2. Input delay says *when data arrives* at our inputs (how much period is already consumed externally before it reaches us); output delay says *when data must leave* our outputs (how much period the external side needs after it). Together they define how much of the period is available for our internal logic on boundary paths.

Q9. What is a timing exception, and can you name the main ones?

Why the interviewer asks: Basic awareness of exceptions.

Expected answer: A timing exception tells the tool that a path should *not* be timed as an ordinary single-cycle path — it's an exception to the default rule that every path must make it from launch to the next capture edge. The main ones: **(1) False path** (`set_false_path`) — the path is not a real functional timing path (e.g. between asynchronous clocks, or a static configuration signal), so don't check it at all. **(2) Multicycle path** (`set_multicycle_path`) — the path is allowed more than one clock cycle to propagate (e.g. a slow datapath whose result is used every other cycle). **(3) Min/max delay** (`set_min_delay` / `set_max_delay`) — an explicit delay requirement on a path, overriding the default clock-derived one. Exceptions matter because real designs have paths that the default single-cycle assumption gets wrong: timing a false path produces meaningless violations, and failing to relax a genuine multicycle path produces false violations. So exceptions correct the default timing to match the real functional intent — but they must be applied carefully, because a wrong exception can hide a real violation.

Follow-up questions: 1. What does a false path tell the tool to do? 2. Why can a wrong exception be dangerous?

Follow-up answers: 1. That the path is not a real functional timing path and should not be timed/checked at all — so the tool ignores it, producing no slack or violation for it (used for things like async crossings or static signals that would otherwise show meaningless violations). 2. Because an exception changes or removes checking on a path — a wrongly-applied false path (or too-broad multicycle) can *hide a real violation* by telling the tool not to check a path that actually needs checking, and since no violation appears, the missed timing problem is silent.

Q10. What are design-rule constraints like `set_max_transition` and `set_max_capacitance`?

Why the interviewer asks: Tests awareness of the environment/design-rule category.

Expected answer: Design-rule constraints set limits that the design must respect regardless of timing slack, reflecting electrical and reliability requirements of the technology. `set_max_transition` limits the slew (transition time) of signals — a too-slow edge degrades the next stage's delay, increases power, and can cause reliability issues, so the library/technology imposes a maximum. `set_max_capacitance` limits the load a driver can drive — an overloaded driver produces slow edges and may be unreliable. There's also `set_max_fanout` (limiting how many loads a net drives). These are called *design rules* because they're constraints on the physical/electrical quality of the implementation, not on functional timing per se — a path can have positive slack but still violate max transition, and that violation must be fixed (usually by buffering or upsizing). Synthesis and optimization treat these as hard limits to respect. They matter because meeting timing isn't

enough: the design must also be electrically well-formed (sharp enough edges, reasonable loads), which the design-rule constraints enforce.

Follow-up questions: 1. Can a path meet timing but still violate a design rule? 2. Why does the technology impose a maximum transition limit?

Follow-up answers: 1. Yes — design rules (max transition, max capacitance, max fanout) are separate from timing slack; a path can have positive timing slack yet still violate, say, max transition (too slow an edge), and that design-rule violation must still be fixed (e.g. by buffering or upsizing). 2. Because a too-slow transition (slew) degrades the receiving stage's delay, increases short-circuit power, and can cause electrical reliability problems; limiting the maximum transition keeps edges sharp enough for reliable, timing-predictable operation.

Q11. What is the difference between `set_input_delay` with `-max` versus `-min` ?

Why the interviewer asks: Tests understanding that I/O delays have both facets.

Expected answer: An input (or output) delay has two facets because setup and hold need different worst cases. The `-max` value is the *latest* the data arrives (largest external delay), used for **setup** analysis — it's the worst case for setup because late-arriving data has the least time to meet the capture edge. The `-min` value is the *earliest* the data arrives (smallest external delay), used for **hold** analysis — it's the worst case for hold because early-arriving data is most likely to change too soon after the capture edge and violate hold. So you typically specify both: `set_input_delay -max` for the setup check and `set_input_delay -min` for the hold check, reflecting that the real external data has a *range* of arrival times, and the two checks are stressed by opposite ends of that range. A common bug is specifying only `-max` (setup) and forgetting `-min` (hold), which leaves hold at the boundary unchecked or defaulted — a silent gap. Correct boundary constraints give both facets.

Follow-up questions: 1. Which facet (`-max` / `-min`) is used for setup, and which for hold? 2. What's the risk of specifying only `-max` ?

Follow-up answers: 1. `-max` (latest arrival, largest external delay) is used for setup — late data is the setup worst case; `-min` (earliest arrival, smallest external delay) is used for hold — early data is the hold worst case. 2. Hold timing at the boundary may be left unchecked or defaulted — specifying only the max (setup) facet and forgetting the min (hold) facet is a silent gap that leaves the boundary hold check missing or wrong, a common I/O-constraint bug.

Q12. What is `set_driving_cell` and why is it used?

Why the interviewer asks: Tests understanding of modeling the input drive.

Expected answer: `set_driving_cell` models the external cell that drives one of our input ports, so the tool knows the realistic edge rate (slew) arriving at the port and can compute the first internal cell's delay accurately. An input port by default might be assumed to have an ideal (infinitely sharp) driver, which is optimistic — real external drivers have finite strength and produce a finite slew, and the first cell's delay depends on that input slew. By specifying `set_driving_cell` (naming a library cell that represents the external driver), the tool applies that cell's drive characteristics to the port, producing a realistic input transition. The alternative is `set_input_transition`, which directly specifies the slew value. Either way, the goal is to avoid the optimistic "ideal edge" assumption and time the input path with a realistic edge. It matters because an unrealistically sharp input edge makes the input path look faster than it really is — an optimistic error — so modeling the real drive is part of accurate boundary timing.

Follow-up questions: 1. What's the risk of not specifying an input drive? 2. What's the alternative to `set_driving_cell` for specifying input edge rate?

Follow-up answers: 1. The tool may assume an ideal, infinitely sharp input edge, which is optimistic — the first internal cell's delay (which depends on input slew) is underestimated, making the input path look faster than reality, an optimistic timing error. 2. `set_input_transition`, which directly specifies the transition (slew) value at the port instead of deriving it from a named driving cell — both achieve a realistic input edge rate, one via a cell model, the other via an explicit slew number.

Q13. What does `set_load` do?

Why the interviewer asks: The output-side environment counterpart.

Expected answer: `set_load` specifies the capacitance an output port must drive — the external load (the capacitance of the board trace, the external device's input pin, etc.) that our output driver sees. This matters because a driver's delay and output slew depend on its load, so to time the path from the launching flop to the output port accurately, the tool needs to know the real external load. Without it, the tool might assume little or no load, making the output path optimistically fast. By specifying `set_load <cap> [get_ports data_out]`, the output path is timed against the realistic load. It's the output-side environment constraint, analogous to how `set_driving_cell` handles the input side: both replace an optimistic default (ideal drive / no load) with a realistic external condition, so boundary paths are timed accurately. It matters because underestimating output load is an optimistic error — the real driver is slower into the real load than into an assumed light one.

Follow-up questions: 1. Why does output load affect the timing of an output path? 2. What optimistic error occurs if output load isn't specified?

Follow-up answers: 1. Because a driver's delay and output slew depend on the capacitance it drives — a heavier load makes the driver slower and its edge softer — so the delay of the path ending at the output port depends on the external load the port must drive. 2. The tool may assume little or no external load, making the output driver appear

faster than it is into the real (heavier) load — an optimistic underestimate of the output path delay.

Q14. What is the purpose of operating conditions in constraints?

Why the interviewer asks: Tests awareness of PVT context.

Expected answer: Operating conditions specify the process, voltage, and temperature (PVT) the design is analyzed under, because cell and wire delays depend strongly on all three. A single design must work across a range of conditions (fast/slow silicon, a supply range, a temperature range), and delays can vary by roughly 2x across that range, so timing must be checked at the relevant corners. Setup is analyzed at the *slow* condition (longest delays, hardest to arrive in time) and hold at the *fast* condition (shortest delays, easiest to race through). The operating conditions (often selected via the library corner and commands like `set_operating_conditions`, or in modern flows via a multi-corner setup) tell the tool which PVT to use for a given analysis. They matter because timing verified at only one nominal condition is meaningless for a part that ships across the whole PVT range — the design must meet setup at the worst slow corner and hold at the worst fast corner, so the operating conditions define which reality each check is stressed against.

Follow-up questions: 1. At which condition is setup analyzed, and at which is hold? 2. Why isn't nominal-condition timing sufficient?

Follow-up answers: 1. Setup is analyzed at the slow condition (slow process, low voltage, the temperature giving longest delays) — late data is the worry; hold at the fast condition (fast process, high voltage, temperature giving shortest delays) — early data is the worry. 2. Because a manufactured part operates across the whole PVT range, not just nominal, and delays vary substantially (roughly up to ~2x) across that range; a path fine at nominal can fail at a slow corner (setup) or fast corner (hold), so timing must be verified at the worst relevant corners, not just nominal.

Q15. What does it mean that "the tool does exactly what the constraints say"?

Why the interviewer asks: Tests the core mindset about constraint responsibility.

Expected answer: It means the timing tool has no independent knowledge of design intent — it only knows what the constraints tell it, and it applies them literally. If you define a clock at the wrong period, it times against that wrong period. If you forget a clock, it doesn't time those paths. If you write a false path that's actually real, it stops checking a path that needs checking. The tool won't second-guess or correct you; it does precisely what's written. The profound consequence is that *constraint errors become timing errors*, and the dangerous ones are silent: a missing constraint produces no violation (nothing is checked), so the report looks clean while real timing is unverified. This is why constraints are the largest source of real timing bugs — not tool inaccuracy, but the tool faithfully

executing wrong or incomplete instructions. The mindset it instills is that the engineer is responsible for the constraints being correct and complete, that a clean report is necessary but not sufficient, and that constraints must be actively verified, not assumed correct because the tool ran.

Follow-up questions: 1. Why does this make constraint errors so dangerous? 2. Why is a clean timing report not proof of correctness?

Follow-up answers: 1. Because the tool applies constraints literally with no notion of intent, a constraint error directly becomes a timing error — and the dangerous ones are silent: a missing or wrong constraint often produces *no* violation (the path isn't checked, or is checked wrongly), so the error doesn't announce itself. 2. Because a clean report only means the paths the tool *was told to check* passed — it says nothing about paths left unchecked by a missing clock or an over-broad false path; the absence of violations can reflect absence of checking, not correctness, so the constraints themselves must be verified.

Q16. What is the relationship between the clock period and the maximum operating frequency?

Why the interviewer asks: Basic but fundamental quantitative relationship.

Expected answer: The clock period and frequency are reciprocals: $\text{frequency} = 1 / \text{period}$. The clock period defined in `create_clock` is the cycle time, and it sets the setup budget — a single-cycle path has one period to propagate from launch to capture. So the shortest period the design can meet (with all setup slack ≥ 0) determines the maximum operating frequency: if the worst setup path needs at least T to close, then the minimum period is T and the maximum frequency is $1/T$. This is why timing closure is fundamentally about making the critical path fit within the period — the critical path's delay sets the floor on the period and thus the ceiling on frequency. For example, a 10 ns period is 100 MHz; if the critical path needs 12 ns, the design can't run at 100 MHz and must either be sped up (optimization, pipelining) or run slower (125 MHz $\rightarrow$ 8... no, 12 ns $\rightarrow$ ~83 MHz). So the period in the SDC directly expresses the target frequency, and meeting setup at that period is what makes the frequency achievable.

Period sets the setup budget; frequency = 1/period



Worked example:

```

period 10 ns -> frequency = 1/10ns = 100 MHz
critical path needs 12 ns -> cannot meet 10 ns period
  either: optimize path to <= 10 ns (keep 100 MHz)
  or:      relax period to 12 ns -> ~83 MHz

```

Follow-up questions: 1. What is the frequency of a 5 ns clock? 2. If the critical path needs more than the period, what are the options?

Follow-up answers: 1. $1 / 5 \text{ ns} = 200 \text{ MHz}$. 2. Either speed up the critical path so it fits the period (logic optimization, resizing, restructuring, or pipelining to break it into shorter stages), keeping the target frequency; or relax the period (lower the frequency) so the path fits — a design/spec trade-off.

Q17. Why must the same SDC be consistent across synthesis, place-and-route, and signoff?

Why the interviewer asks: Tests understanding of cross-stage consistency.

Expected answer: Because the three stages must all be working toward and verifying against the *same* timing target, or they disagree. Synthesis builds the netlist to meet the SDC's budgets; P&R optimizes placement/routing against the SDC; signoff verifies the final result against the SDC. If the SDC differs between stages — a constraint present at signoff but not at synthesis, a period that changed, an exception applied in one flow but not another — then synthesis might build to a target that signoff doesn't check (wasted or misdirected effort), or signoff might check a target the design was never built for (unexpected violations). The design can pass one stage and fail the next purely because the constraints drifted, not because of any real timing change. This is why teams strive for a *single source* of constraints propagated consistently to all stages, rather than separately maintained SDC files that can diverge. Consistency ensures the stages form a coherent chain — built to the same intent that's finally verified — so closure at signoff reflects the real design, not a constraint mismatch.

Follow-up questions: 1. What can happen if a constraint exists at signoff but not at synthesis? 2. How do teams avoid SDC drift between stages?

Follow-up answers: 1. Signoff checks a requirement the design was never built to meet — synthesis didn't optimize for it — so the path may show an unexpected violation at signoff that could have been prevented had synthesis known about the constraint; the stages were working to different targets. 2. By maintaining a *single source* of constraints (one authoritative SDC or a generator) that's propagated consistently to synthesis, P&R, and signoff — rather than separate, independently-edited SDC files per stage that can diverge — plus checks that the stages' constraints match.

Q18. What is a virtual clock and when is it used in I/O constraints?

Why the interviewer asks: Connects clocks to I/O constraints at a basic level.

Expected answer: A virtual clock is a clock defined with no source object — it exists purely as a timing reference, not driving any real pin in the design. It's created with `create_clock -name VCLK -period <P>` and no port/pin argument. Its main use is in I/O constraints: it models an *external* clock that times the outside world but doesn't physically exist inside our design. When we say "data arrives at this input port 3 ns after the external clock's edge," that external clock may not be a real net in our design — so we define it as a virtual clock and reference the input/output delay to it (`set_input_delay 3 -clock VCLK [get_ports data_in]`). This cleanly separates the external timing reference from our internal clocks. It's used whenever the I/O timing is relative to a clock that isn't one of our internal clocks — commonly for interfaces where the external device has its own clock. So virtual clocks are the standard way to give I/O delays a proper external reference without inventing a fake physical clock net.

Follow-up questions: 1. What distinguishes a virtual clock from a normal clock? 2. Why use a virtual clock for I/O timing?

Follow-up answers: 1. A virtual clock has no source object — it's defined with `create_clock` and a name/period but no port or pin — so it doesn't drive any physical net; it exists only as a timing reference, whereas a normal clock is attached to a real source pin/port. 2. To provide a proper external timing reference for input/output delays when the relevant clock isn't one of our internal clocks — it models the external device's clock without inventing a fake physical clock net, cleanly separating the external reference from internal clocks.

Q19. What information does a basic `report_timing` give you?

Why the interviewer asks: Tests familiarity with the primary output.

Expected answer: `report_timing` shows the detailed breakdown of a timing path — by default the most critical (worst-slack) one, or a specified path. It shows: the **start point** and **end point** (where the path launches and captures); the **launch clock path** and **capture clock path** (the clock arrival at each end); the **data path** broken down arc by arc (each cell and net with its incremental delay, and often transition and load); the **arrival time** (when the data actually gets to the endpoint) and the **required time** (when it needs to get there, from the clock and constraints); and the **slack** (required – arrival for setup), the bottom line saying whether the path passes (slack ≥ 0) or fails. It also indicates which check (setup or hold) and the clocks involved. Reading it lets you see not just *that* a path fails but *why* — which arcs dominate the delay, whether it's a cell or net problem, and how much margin there is. It's the primary tool for diagnosing timing, so fluency in reading it is essential.

Follow-up questions: 1. What is slack, and what does its sign tell you? 2. What does the arc-by-arc data path breakdown help you diagnose?

Follow-up answers: 1. Slack is required time minus arrival time (for setup) — the margin by which the path passes or fails; positive slack means it meets timing with room to spare, negative slack means it violates (arrives later than required), and its magnitude is how much. 2. It shows each cell and net's incremental delay along the path, so you can see which arcs dominate — whether the delay is concentrated in cell arcs (a logic/sizing issue) or net arcs (a routing/interconnect issue), and where to focus a fix.

Q20. Why is constraint development considered as important as RTL design?

Why the interviewer asks: Tests appreciation of constraints' importance — a mindset question.

Expected answer: Because the constraints determine whether the timing analysis — and thus the whole timing signoff — is meaningful, and errors in them are as consequential as errors in the RTL. Correct RTL with wrong constraints yields a design that's verified against the wrong requirements: it can be built and signed off "clean" yet fail in silicon because the constraints didn't reflect reality (wrong period, missing clock, wrong exception hiding a real violation). The constraints are the specification of *how the design must behave in time*, and STA only guarantees the design meets whatever the constraints say — so if the constraints are wrong, the guarantee is worthless. Moreover, constraint errors are often silent (a missing constraint leaves paths unchecked with no violation), making them harder to catch than a functional RTL bug that a simulation might expose. So constraint development demands the same rigor, review, and verification as RTL: it's not clerical setup, it's the encoding of the timing specification, and the design's real-world timing correctness depends on it every bit as much as on the logic itself. Treating constraints as an afterthought is how good RTL ends up in a failing chip.

Follow-up questions: 1. What happens with correct RTL but wrong constraints? 2. Why are constraint errors often harder to catch than RTL bugs?

Follow-up answers: 1. The design is verified against the wrong timing requirements — it can be built and signed off looking clean, yet fail in silicon, because STA only guarantees the design meets what the constraints say, and wrong constraints (wrong period, missing clock, false exception) mean a meaningless guarantee. 2. Because constraint errors are often silent — a missing clock or over-broad false path produces *no* violation (the path simply isn't checked), so nothing flags the problem — whereas a functional RTL bug often shows up as a simulation mismatch or failing test; the absence of a symptom makes constraint errors easy to miss until silicon.

Level 2 — Intermediate (2-5 Years)

Q1. Walk through the order in which you would write an SDC file from scratch, and why.

Why the interviewer asks: Tests practical, dependency-aware constraint authoring.

Expected answer: I'd follow the dependency order, because SDC is Tcl evaluated top to bottom and later commands reference earlier objects. **(1) Setup/units and library** context first (if not already established). **(2) Clocks** — primary clocks (`create_clock`), then generated clocks (which reference their source clocks), then clock latency, uncertainty, and (stage-appropriate) propagated/ideal state. Clocks come first because everything references them. **(3) Clock relationships** — asynchronous and exclusive clock groups, so cross-domain paths are correctly timed or excluded. **(4) I/O constraints** — input/output delays (referenced to the now-defined clocks, real or virtual), then the drive/load environment (`set_driving_cell` / `set_input_transition`, `set_load`). **(5) Timing exceptions** — false paths, multicycle paths, min/max delays, applied after the base timing is set so they modify a correct baseline. **(6) Design-rule and environment** — max transition/capacitance/fanout, operating conditions. The rationale is that each layer depends on the previous: I/O delays need clocks to exist; exceptions modify paths that need clocks and I/O defined; and putting exceptions last makes their overriding effect predictable. Following this order avoids "object doesn't exist yet" errors and makes the file's logic clear and reviewable.

Follow-up questions: 1. Why do clocks come before I/O delays in the file? 2. Why are exceptions typically placed after the base constraints?

Follow-up answers: 1. Because I/O delays are referenced to a clock (`-clock`), so the clock object must already exist when the I/O delay command runs; more generally clocks are the foundation everything else references, so they're defined first. 2. So they modify a correct, fully-defined baseline — exceptions override default timing on specific paths, and placing them after the clocks, I/O, and relationships are set makes their overriding effect predictable and ensures the paths they reference are already properly constrained.

Q2. How does synthesis use the clock period to make optimization decisions?

Why the interviewer asks: Tests understanding of timing-driven synthesis.

Expected answer: The clock period is the setup budget synthesis works against, and it drives nearly every timing-related decision. Given the period, synthesis knows how much delay can fit between flops, so it: **(1)** chooses how to map logic to cells — using faster (larger, higher-drive) cells on paths that are tight against the period, and smaller/lower-power cells where there's slack; **(2)** decides structural transformations — restructuring logic to reduce levels on critical paths, or leaving less-critical logic alone; **(3)** sizes and buffers to meet the budget while respecting design rules; **(4)** allocates effort — spending

optimization effort proportional to how close a path is to violating the period. A tighter (shorter) period pushes synthesis toward faster, larger, more power-hungry implementations; a looser period lets it optimize for area/power. If the period is wrong — too loose — synthesis under-optimizes and the design won't actually meet the real frequency; too tight, it over-optimizes, wasting area and power. So the period isn't just a check value; it's the primary knob steering how aggressively synthesis builds the netlist, which is why getting it right at synthesis (consistent with signoff) matters from the very first step.

Follow-up questions: 1. How does a tighter period change synthesis's choices? 2. What happens if synthesis is given a too-loose period?

Follow-up answers: 1. A tighter period pushes synthesis toward faster implementations — larger, higher-drive cells, more aggressive logic restructuring to reduce levels, and more buffering on critical paths — at the cost of more area and power, because it has less time budget to fit the logic into. 2. Synthesis under-optimizes — it sees plenty of budget, so it uses smaller/slower cells and less restructuring; the resulting netlist won't meet the real (tighter) frequency, and the problem surfaces downstream when signoff checks against the correct period.

Q3. What is the difference between `set_max_delay` / `set_min_delay` and the clock-derived timing?

Why the interviewer asks: Tests understanding of explicit path constraints vs default.

Expected answer: By default, a path's timing requirement is *derived from the clocks* — a single-cycle flop-to-flop path must go from launch to the next capture edge (one period, adjusted for setup/skew/uncertainty). `set_max_delay` and `set_min_delay` instead impose an *explicit* delay requirement on a path, overriding that clock-derived default. `set_max_delay <value> -from ... -to ...` says the path delay must be $\leq$ value regardless of the clock relationship; `set_min_delay` sets a floor. They're used where the clock-derived timing doesn't apply or isn't what you want: purely combinational paths (input port to output port, with no clock to derive a budget), asynchronous interfaces where you want to bound the delay without a clock relationship, or specific paths needing a custom requirement. They're technically also timing exceptions (they override the default). The key distinction is *source of the requirement*: clock-derived timing comes from the clock waveforms automatically, while max/min delay is a manual, explicit number you impose. Because they override, they must be used carefully — an over-broad `set_max_delay` could relax paths that should be timed by the clock. They're the tool for when you need to state a path's requirement directly rather than let the clocks define it.

Follow-up questions: 1. When would you use `set_max_delay` instead of relying on clock-derived timing? 2. Why must `set_max_delay` be used carefully?

Follow-up answers: 1. For paths where clock-derived timing doesn't apply or isn't desired — purely combinational input-to-output paths (no clock to derive a budget), asynchronous interfaces where you want to bound delay without a clock relationship, or a specific path needing a custom explicit requirement. 2. Because it overrides the default clock-derived

timing on the paths it covers — an over-broad `-from/-to` scope could relax paths that should be timed against the clock, silently replacing a correct clock-derived check with a looser (or different) explicit one, potentially hiding a real requirement.

Q4. How do input and output delays relate to the external device's timing?

Why the interviewer asks: Tests the physical meaning behind I/O delay numbers.

Expected answer: The I/O delay numbers encode the external device's timing so our internal timing is budgeted correctly for the shared interface. For an **input**, the data comes from an external device that launches it on the (shared or related) clock edge; `set_input_delay` = the external device's clock-to-output delay + the interconnect delay from it to our port. That's how much of the clock period is consumed *before* the data reaches us, so our internal path has (period – input_delay – our_setup) to work with. For an **output**, the data goes to an external device that captures it; `set_output_delay` = the external device's setup requirement + the interconnect delay from our port to it. That's how much of the period the external side needs *after* our data leaves, so our internal path must deliver by (period – output_delay). So the I/O delays are literally the external device's parameters (clock-to-Q, setup) plus board/interconnect delay, translated into "how much period is used outside our design." Getting them right requires knowing the external device's datasheet timing and the interconnect — they're a shared budget, and an error (guessing wrong external timing) directly mis-budgets our internal paths.

Worked example:

```
Input: external clk-to-Q = 2 ns, board delay = 1 ns
-> set_input_delay 3 (data valid 3 ns after clock edge at our port)
    our internal budget on that path = period - 3 - our_setup

Output: external setup = 1.5 ns, board delay = 0.5 ns
-> set_output_delay 2 (external side needs data 2 ns before its clock edge)
    our internal path must deliver by period - 2
```

Follow-up questions: 1. What two components make up an input delay value? 2. What two components make up an output delay value?

Follow-up answers: 1. The external launching device's clock-to-output (clock-to-Q) delay plus the interconnect/board delay from that device to our input port — together, how much of the clock period is consumed before the data arrives at us. 2. The external capturing device's setup requirement plus the interconnect/board delay from our output port to that device — together, how much of the clock period the external side needs after the data leaves our port.

Q5. What is `set_case_analysis` and when is it used?

Why the interviewer asks: Tests knowledge of mode/constant setting.

Expected answer: `set_case_analysis` sets a constant logic value (0 or 1) on a pin or port, telling the tool that signal is held at that value for the analysis. The tool then propagates the constant, which disables paths through logic that the constant makes inactive and selects specific modes/configurations. Uses: **(1) Mode selection** — setting a mode-control signal to select functional vs test mode, or one configuration of a multi-function block, so only the active mode's paths are timed. **(2) Clock/path selection** — setting a mux select so only one clock or datapath propagates (e.g. selecting one clock through a clock mux). **(3) Disabling unused logic** — constants that turn off paths not relevant to the current analysis. By fixing these control signals, `set_case_analysis` makes the timing analysis reflect a specific operating configuration rather than timing all structurally-possible paths (many of which are mutually exclusive by mode). It's essential for multi-mode designs — each mode is analyzed with the appropriate case-analysis settings selecting it. The benefit is accuracy and reduced pessimism: without it, the tool might time paths that can't coexist (different modes' paths simultaneously), producing false violations; with it, only the real, active-configuration paths are timed.

Follow-up questions: 1. How does `set_case_analysis` reduce false violations? 2. Give an example use of `set_case_analysis`.

Follow-up answers: 1. By fixing control signals to a specific configuration, it disables paths through logic the constants make inactive — so the tool only times the paths real in that configuration, rather than timing mutually-exclusive paths from different modes simultaneously (which would produce false violations from combinations that can't actually occur). 2. Setting a mode-select signal to choose functional vs test mode (timing only that mode's paths), or setting a clock-mux select constant so only one clock source propagates to the downstream domain for that analysis — selecting a specific, real operating configuration.

Q6. How would you constrain a purely combinational path from an input port to an output port?

Why the interviewer asks: Tests handling of paths with no flop endpoints.

Expected answer: A combinational path from input port to output port has no launching or capturing flop, so there's no clock to derive its budget from — the clock-derived timing doesn't directly apply. There are two common approaches. **(1)** If the combinational path is part of a clocked interface (data flows through it between clocked devices), constrain it with input and output delays referenced to the interface clock — `set_input_delay` at the input port and `set_output_delay` at the output port — so the path is budgeted within the period like any I/O path, just with no internal flop. The combined external timing plus the required internal delay must fit the period. **(2)** If it's a standalone combinational requirement (e.g. a pure delay spec through the block), use `set_max_delay` (and

`set_min_delay` for hold-like) to impose an explicit delay budget directly on the port-to-port path. The choice depends on whether the path lives within a clocked interface budget (use I/O delays) or needs a direct delay bound (use max/min delay). Either way, the point is that combinational port-to-port paths need *explicit* constraint (I/O delays or max/min delay) because there's no clock relationship to budget them automatically — leaving them unconstrained is a common gap.

Follow-up questions: 1. Why doesn't clock-derived timing directly apply to an input-to-output combinational path? 2. What are the two ways to constrain such a path?

Follow-up answers: 1. Because there's no launching or capturing flip-flop on the path — clock-derived timing budgets a path from a launch clock edge to a capture clock edge, but a pure input-to-output combinational path has neither flop, so there's no clock relationship to derive a budget from. 2. Either constrain it with input and output delays referenced to the interface clock (budgeting it within the period like an I/O path), or impose an explicit `set_max_delay` (and `set_min_delay`) directly bounding the port-to-port delay — depending on whether it's part of a clocked interface or a standalone delay requirement.

Q7. What is the difference between `set_disable_timing` and `set_false_path`?

Why the interviewer asks: Tests a subtle distinction between two "don't time" mechanisms.

Expected answer: Both stop timing, but at different granularity and for different reasons. `set_false_path` is a *timing exception* that tells the tool a specific *path* (or set of paths, via `-from` / `-through` / `-to`) is not a real functional timing path and shouldn't be checked — it operates at the path level and is about *functional intent* (this path can't/won't happen functionally). `set_disable_timing` disables a specific *timing arc* within a cell (e.g. a particular input-to-output arc of a gate) — it's more surgical, removing that arc from all timing, and is about the *arc* being invalid or unwanted (e.g. a arc that shouldn't be used, or breaking a combinational loop). So false path = "don't time this path (functional intent)"; disable timing = "don't use this specific cell arc at all (structural)." A common use of `set_disable_timing` is breaking a combinational feedback loop by disabling one arc so the tool can analyze the rest; false paths handle functional exceptions like async crossings. Confusing them leads to using the wrong tool — disabling an arc when you meant to except a path (over-broad, affects all paths through that arc) or vice versa. The distinction is granularity (arc vs path) and intent (structural invalidity vs functional exception).

Follow-up questions: 1. What granularity does each operate at — arc or path? 2. Give a typical use of `set_disable_timing`.

Follow-up answers: 1. `set_disable_timing` operates at the *arc* level (disabling a specific input-to-output timing arc within a cell); `set_false_path` operates at the *path* level (excepting a whole path or set of paths from timing). 2. Breaking a combinational feedback loop — disabling one arc in the loop so the tool can time the rest of the logic (a loop otherwise can't be statically analyzed); also disabling a specific invalid or unused cell arc so it isn't considered in any path.

Q8. How do you check that your constraints actually applied to the intended objects?

Why the interviewer asks: Tests the verification discipline against silent no-ops.

Expected answer: Since a query matching nothing silently applies a constraint to zero objects, I verify the queries and the applied constraints rather than trusting a clean run. Methods: **(1)** Evaluate the object queries directly — run `get_ports`, `get_pins`, `get_clocks`, `get_cells` with the intended pattern and check they return the expected objects (a non-empty, correct collection); many tools warn on empty queries if you enable that. **(2)** Use report commands — `report_clocks` (clocks defined), `report_port` / `report_timing` with the relevant endpoints (to see if I/O delays and exceptions took effect), `check_timing` (which flags unconstrained/missing constraints), and exception-reporting commands to confirm false/multicycle paths applied to the intended paths. **(3)** `check_timing` specifically reports unconstrained endpoints, missing clocks, and other gaps — a key sanity check. **(4)** Review the log for warnings about empty collections or unapplied commands. The discipline is to treat constraint application as something to *confirm*: did this clock get defined, did this I/O delay attach to the right port, did this false path cover the intended paths, and are there any unconstrained endpoints? Because the failure mode is silent, active verification (queries return what's expected, `check_timing` is clean of unconstrained paths, reports confirm application) is essential — a clean timing run alone doesn't prove the constraints applied as intended.

Follow-up questions: 1. What does `check_timing` help you find? 2. Why isn't a clean timing run enough to confirm constraints applied?

Follow-up answers: 1. `check_timing` reports constraint gaps and problems — unconstrained timing endpoints, missing clocks, undefined I/O delays, and similar issues — flagging paths that aren't properly constrained (which wouldn't otherwise show as violations because they're not being checked). 2. Because a constraint whose query matched nothing runs without error and simply applies to zero objects — so the run is clean, but the intended paths are unconstrained; the clean run reflects the tool doing what it was (mis)told, not that the constraints applied correctly, so queries and application must be actively verified.

Q9. What is the role of `set_clock_groups` versus `set_false_path` for handling unrelated clocks?

Why the interviewer asks: Tests preferred method for clock-domain exclusion (revisits from the SDC-structure angle).

Expected answer: Both can stop timing between clock domains, but `set_clock_groups` is the cleaner, preferred method for declaring clock *relationships*. `set_clock_groups -asynchronous -group {CLKA} -group {CLKB}` declares the two clocks asynchronous, so the tool doesn't time paths between them *in either direction*, automatically and comprehensively — including future paths. `set_false_path -from [get_clocks CLKA] -to [get_clocks CLKB]`

achieves the same for that one direction, but you'd need the reverse too, and it's a path exception rather than a relationship declaration. The advantages of clock groups: it's a single, symmetric, intent-revealing statement of the *relationship* (these clocks are async), it covers both directions and all paths automatically, and it's less error-prone than maintaining pairwise false paths. It also supports exclusivity (`-physically_exclusive` , `-logically_exclusive`) for clocks never active together. So the guidance is: use `set_clock_groups` to declare clock relationships (async/exclusive) — it's the correct tool for domain-level relationships — and reserve `set_false_path` for specific path exceptions that aren't about a whole clock relationship. Using false paths for what should be a clock-group relationship works but is more verbose and easier to get incomplete.

Follow-up questions: 1. Why is `set_clock_groups` preferred over pairwise false paths for async clocks? 2. When would you still use `set_false_path` rather than clock groups?

Follow-up answers: 1. Because it's a single, symmetric declaration of the *relationship* that automatically covers both directions and all paths between the domains (including future ones), whereas pairwise false paths must be written per direction and are easier to leave incomplete — clock groups are cleaner, comprehensive, and reveal the async intent. 2. For a specific *path* exception that isn't about a whole clock-domain relationship — e.g. excepting one particular path (or a small set via `-from/-through/-to`) that's functionally false, where the exclusion is path-specific rather than a statement that two entire clock domains are unrelated.

Q10. How does an incorrect input delay affect timing, and in which direction?

Why the interviewer asks: Tests understanding of I/O delay error consequences.

Expected answer: An input delay budgets how much of the period is consumed externally before data reaches our port, so an error mis-budgets the internal path. If the input delay is set *too small* (claiming data arrives earlier than it really does), the tool thinks the internal path has *more* time than it actually does — an **optimistic** error: it may pass a path that will really fail, because in reality the data arrives later and the internal logic has less time. This is dangerous (a real setup failure at the boundary goes undetected). If the input delay is set *too large* (claiming data arrives later than it does), the tool thinks the internal path has *less* time — a **pessimistic** error: it may fail a path that would really pass, causing wasted optimization effort or over-design, but it's safe. So the dangerous direction is *too-small* input delay (optimistic, hides real failures), and the safe-but-wasteful direction is *too-large* (pessimistic). The same logic applies with the appropriate sign to the min facet for hold. The practical lesson: err toward accuracy, but if uncertain, a slightly conservative (larger max) input delay is safer than an optimistic (too-small) one — and the value should come from the real external device timing, not a guess, because a wrong input delay directly translates to a wrong internal budget in a specific, knowable direction.

Follow-up questions: 1. Which direction of input-delay error is optimistic and dangerous? 2. Why is a too-large input delay safe but wasteful?

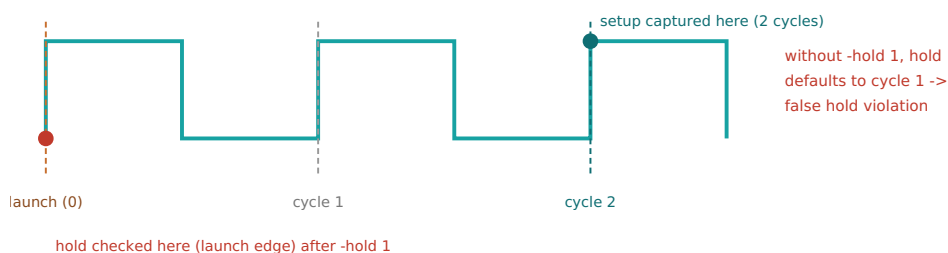
Follow-up answers: 1. Too *small* an input delay (claiming the data arrives earlier than it really does) is optimistic and dangerous — the tool thinks the internal path has more time than it really has, so it can pass a boundary path that will actually fail setup in silicon, hiding a real failure. 2. Because a too-large input delay makes the tool think the internal path has *less* time than it really does (pessimistic) — it may flag violations that wouldn't really occur, causing wasted optimization or over-design, but it won't hide a real failure, so it's safe (just inefficient).

Q11. What does `set_multicycle_path` do, and why is the hold multiplier important?

Why the interviewer asks: Tests the classic multicycle setup/hold subtlety.

Expected answer: `set_multicycle_path` relaxes a path's timing to allow more than one clock cycle to propagate — used when a path is functionally allowed multiple cycles (e.g. a slow datapath whose result is used every N cycles). `set_multicycle_path N -setup` moves the setup capture edge out to the Nth cycle, giving the path N periods instead of one. The subtlety is the *hold* check: by default, specifying `-setup N` leaves the hold check referenced to the cycle *before* the moved setup edge (cycle N-1), which demands the data be held stable for many cycles — an unrealistic requirement that produces a *false hold violation*. The fix is to also specify the hold multiplier, `set_multicycle_path N-1 -hold` (typically), which moves the hold check back to the launch edge where it belongs, so hold is checked normally (same-edge). Forgetting the hold multiplier is one of the most common and dangerous multicycle mistakes — it either produces confusing false hold violations or, if mis-set, can create a real hold problem. So the rule is: a multicycle setup almost always needs its companion hold multiplier to keep the hold check at the correct edge. Understanding *why* (the default hold edge follows the moved setup edge) is what distinguishes someone who can safely apply multicycle paths.

Multicycle: -setup 2 moves capture out; -hold 1 fixes the hold edge



Worked example:

```
set_multicycle_path 2 -setup    ;# setup captured at cycle 2 (2 periods)
    default hold edge -> cycle 1 (demands hold across a full cycle -- WRONG)
set_multicycle_path 1 -hold     ;# move hold back to launch edge (cycle 0) -- CORRECT
```

Follow-up questions: 1. What does `-setup 2` alone do to the hold check? 2. Why does a multicycle setup usually need a companion hold multiplier?

Follow-up answers: 1. It leaves the hold check referenced to the cycle just before the moved setup edge (cycle 1), which demands the data be held stable across a full extra cycle — an unrealistic requirement that produces a false hold violation. 2. Because moving the setup edge out also shifts the default hold edge, creating an over-strict hold requirement; the companion hold multiplier (`-hold N-1`) moves the hold check back to the launch edge where it belongs, so hold is checked correctly (same-edge) — without it, you get false hold violations or a mis-set hold check.

Q12. How do wire-load models or extracted parasitics factor into the constraint/analysis picture?

Why the interviewer asks: Tests understanding of how interconnect enters (esp. at synthesis).

Expected answer: Interconnect delay depends on wire R and C, which aren't known until layout — so at different stages the tool estimates or uses real parasitics. Early (synthesis, pre-layout), there's no real routing, so synthesis uses **wire-load models** (statistical estimates of wire RC as a function of fanout/block size) to approximate net delays — crude but enough to guide optimization. After layout, **extracted parasitics** (SPEF) give the real per-net RC, and signoff STA uses those for accurate net delays. From a constraints perspective, this mostly affects *accuracy*, not the SDC commands themselves — the same clocks/exceptions apply, but the net delays feeding the analysis get more accurate through the flow. The relevance to constraints: at synthesis, over-optimistic wire-load estimates can make synthesis think paths are faster than they'll really be after routing, so the netlist may need rework post-layout (the classic pre- vs post-layout timing gap). Modern flows reduce this by using physical synthesis (placement-aware estimates) instead of statistical wire loads. So wire-load models vs extracted parasitics is about how interconnect delay is estimated vs measured across the flow, and the constraints stay largely the same while the analysis they drive becomes more accurate — with the caveat that early estimates must be reasonable or synthesis optimizes against a wrong interconnect picture.

Follow-up questions: 1. Why does synthesis use wire-load models instead of extracted parasitics? 2. What's the risk of over-optimistic wire-load estimates at synthesis?

Follow-up answers: 1. Because at synthesis there's no real routing yet — the wires don't physically exist — so there are no extracted parasitics to use; wire-load models provide a statistical estimate of net RC (from fanout/block size) to approximate interconnect delay for optimization until layout produces real parasitics. 2. Synthesis may think paths are faster than they'll really be after routing (real wires are slower than the optimistic estimate), so the netlist that looks closed at synthesis fails timing post-layout — the pre- vs post-layout gap — requiring rework; physical (placement-aware) synthesis mitigates this.

Q13. What is the difference between ideal and propagated clocks from a synthesis-vs-signoff standpoint?

Why the interviewer asks: Revisits ideal/propagated in the flow-stage context.

Expected answer: Ideal vs propagated is about whether the clock network's real delays are modeled, and it maps onto the flow stages. During **synthesis** (and early P&R), the clock tree doesn't exist yet, so clocks are **ideal** — the tool assumes the clock arrives at all flops simultaneously (zero skew) and uses an estimated latency and an uncertainty that *includes a skew estimate* to reserve margin for the tree that will be built. This lets synthesis optimize the data paths without a real clock tree. After **clock tree synthesis**, the tree exists, so clocks become **propagated** — the tool computes the real clock arrival at each flop (real insertion delay and real skew), and the uncertainty is reduced (skew is now real in the latency, so only jitter plus margin remains in uncertainty). At **signoff**, propagated clocks with real (extracted) clock-tree timing give the authoritative analysis. So from synthesis to signoff, clocks go from ideal (estimated, zero-skew, skew-in-uncertainty) to propagated (real latency and skew from the built tree). The practical implication: synthesis timing is approximate on the clock side (ideal), so a design "closed" at synthesis can shift once real clock skew appears — which is why post-CTS timing is where clock reality enters and why uncertainty is adjusted at that transition.

Follow-up questions: 1. Why are clocks ideal during synthesis? 2. What changes in the uncertainty when moving from ideal to propagated?

Follow-up answers: 1. Because the clock tree hasn't been built yet at synthesis — there's no real network to propagate delays through — so the tool assumes an ideal clock (arriving everywhere simultaneously, zero skew) with estimated latency, letting it optimize data paths before the tree exists. 2. The skew estimate is removed from uncertainty because real skew is now computed in the propagated network latency — so uncertainty shrinks to mainly jitter plus a modest margin, avoiding double-counting the skew that's now real in the latency.

Q14. How would you constrain a design with two clock domains that exchange data through a dual-clock FIFO?

Why the interviewer asks: Tests practical CDC-structure constraining.

Expected answer: A dual-clock FIFO is a standard CDC structure: the write side is in clock domain A, the read side in domain B, and the FIFO safely passes data between them (with gray-coded pointers synchronized across the domains). Constraining it: **(1)** Define both clocks (`create_clock` for each). **(2)** Declare the two domains asynchronous with `set_clock_groups -asynchronous -group {CLKA} -group {CLKB}`, so the tool doesn't try to time the (meaningless) synchronous relationship between the write and read domains through the FIFO's crossing. **(3)** Within each domain, the FIFO's write logic (in A) and read logic (in B) are timed normally by their respective clocks — the intra-domain paths are real single-cycle paths. **(4)** The pointer synchronizers (the gray-code pointers crossing domains) are

the actual CDC crossing; the async clock-group declaration stops STA from timing them synchronously, and their metastability safety is verified by CDC analysis, not STA. **(5)** Sometimes a `set_max_delay` is applied on the crossing pointer bits to bound their delay/skew. So the constraint essence is: define both clocks, declare them async (don't time the crossing), let each domain's intra-FIFO logic be timed normally, and rely on CDC verification for the synchronizer correctness. The FIFO is the *design* solution to the async crossing; the constraints just tell STA to time each side normally and not the crossing.

Follow-up questions: 1. Why declare the two FIFO clock domains asynchronous? 2. What verifies the FIFO's pointer synchronizers if STA doesn't time the crossing?

Follow-up answers: 1. Because the write and read domains have no fixed phase relationship (async clocks), so timing the crossing between them synchronously is meaningless — declaring them async tells STA not to time the crossing, while each domain's internal FIFO logic is still timed normally by its own clock. 2. CDC (clock-domain-crossing) analysis — it verifies the synchronizers exist on all crossings, are structurally correct (e.g. gray-coded pointers, proper multi-flop synchronizers), and provide adequate metastability settling; STA can't assess metastability, so CDC handles the crossing's safety separately.

Q15. What is the meaning and use of the `-through` option in exceptions?

Why the interviewer asks: Tests precise exception scoping.

Expected answer: The `-through` option scopes an exception (false path, multicycle, max/min delay) to paths that pass *through* a specific point (pin, net, or port), in addition to or instead of `-from` (start) and `-to` (end). It lets you target a *subset* of paths precisely — e.g. `set_false_path -through [get_pins mux/sel]` applies only to paths going through that pin, not all paths between two endpoints. This is useful when the exception should apply only to paths using a particular route (through a specific mux input, a specific functional block, a certain net) rather than all paths between a from/to pair. You can combine `-from`, `-through`, and `-to` to pin down exactly the intended paths, and even use multiple `-through` points to require the path pass through several points. The power and the risk are both in the precision: used well, `-through` scopes an exception to exactly the right paths, avoiding over-broad exceptions that would incorrectly except unintended paths; used carelessly (too broad, or matching more than intended), it can except paths that should be timed. So `-through` is the tool for surgical exception scoping — essential when a from/to pair alone would catch too many paths, and a key part of writing exceptions that cover exactly the intended paths and no others.

Follow-up questions: 1. What does `-through` let you target that `-from`/`-to` alone cannot? 2. Why is precise exception scoping important?

Follow-up answers: 1. Paths passing through a specific intermediate point (pin/net/port) — a *subset* of the paths between a from/to pair that use a particular route — which `-from`/`-to` alone (specifying only endpoints) can't distinguish; `-through` narrows to paths through that point. 2. Because an over-broad exception incorrectly excepts (or relaxes)

paths that should be timed, silently hiding real requirements — precise scoping (via `-through` and exact `-from/-to`) ensures the exception covers exactly the intended paths and no others, so it corrects the right paths without disabling checking on paths that need it.

Q16. How do you handle constraints for a block that will be reused in multiple contexts?

Why the interviewer asks: Tests reusable/portable constraint thinking.

Expected answer: A reusable block (IP) will be integrated into different top-level contexts, so its constraints must be *portable* and *context-independent* where possible, with clear assumptions where not. Approach: **(1)** Write the block's internal constraints (its own clocks as they arrive at its boundary, internal exceptions) in a self-contained way that doesn't hard-code top-level specifics. **(2)** Clearly document the block's *interface assumptions* — what it expects at its boundary (clock frequencies/relationships at its ports, I/O timing budgets it assumes) — so each integration context can supply consistent top-level constraints. **(3)** Parameterize where the block is used at different frequencies or configurations (periods as variables), so the same constraint template adapts. **(4)** Provide a *timing model* (or budget) for the block that the top level uses, and ensure the block's assumptions match what each top delivers (the hierarchical consistency issue). **(5)** Avoid embedding top-specific exceptions in the block's SDC; those belong at the integration level. The goal is that the block's constraints capture its *internal* timing intent portably, its *interface contract* is explicit and checkable, and integration into each context is a matter of supplying matching top-level constraints — so the block times correctly in every context without rewriting its constraints. This mirrors good IP practice: self-contained internal constraints plus a documented, consistent interface contract.

Follow-up questions: 1. What must be documented for a reusable block's constraints? 2. Why avoid embedding top-specific exceptions in the block's own SDC?

Follow-up answers: 1. The block's interface assumptions — the clock frequencies and relationships it expects at its boundary ports, and the I/O timing budgets it assumes — so each integration context can supply consistent top-level constraints that match what the block was designed and closed against. 2. Because top-specific exceptions don't belong to the block's internal intent and won't be correct in every context — embedding them makes the block's SDC non-portable and can wrongly except paths in a context where they should be timed; such exceptions belong at the integration (top) level where the specific context is known.

Q17. What is `group_path` (or path grouping) and how does it help timing closure?

Why the interviewer asks: Tests knowledge of organizing optimization effort.

Expected answer: Path grouping (`group_path`) organizes timing paths into named groups so the tool can report and optimize them separately, with control over effort and cost per group. By default paths are grouped by their capturing clock, but you can create custom groups — e.g. all input paths, all output paths, all paths through a critical block, or register-to-register paths of a domain. Benefits: **(1) Reporting** — you can see the worst slack per group, isolating where the problems are (input paths vs internal vs output). **(2) Optimization control** — synthesis/optimization can be directed to weight groups differently (e.g. prioritize the critical clock's reg-to-reg paths), and the cost function considers each group's worst path, so improving one group doesn't get masked by another. **(3) Closure strategy** — grouping lets you tackle closure group by group and track progress. It helps closure because a flat view of all paths can hide structure (the worst path in one category dominates the report while other categories also have problems); grouping surfaces each category's status and lets optimization spread effort appropriately. So path grouping is an organizational tool that makes large-scale timing closure tractable — separating paths into meaningful groups for targeted reporting and optimization, rather than treating the whole design as one undifferentiated pool of paths.

Follow-up questions: 1. How are paths grouped by default? 2. How does path grouping help optimization?

Follow-up answers: 1. By their capturing clock — each clock domain's paths form a default group — so the tool reports and optimizes per-clock unless you define custom groups (e.g. by input/output/block). 2. It lets optimization consider each group's worst path separately and weight groups by priority, so effort is spread appropriately across categories (input, output, internal, critical block) rather than being dominated by one group's worst path — preventing one category's problems from masking another's and enabling targeted closure.

Q18. How would you approach constraining a design you didn't create, with no existing SDC?

Why the interviewer asks: Tests methodical bring-up of constraints from scratch.

Expected answer: I'd build the constraints methodically from the design's structure and specification, verifying coverage as I go. **(1) Understand the design** — get the spec/datasheet: intended clock frequencies, clock relationships, interfaces and their timing, modes. Examine the netlist for clock sources, sequential elements, and I/O ports. **(2) Define clocks first** — create primary clocks at their sources with the spec frequencies, generated clocks for dividers/PLLs, and declare relationships (async/exclusive) per the design's intent. **(3) Add I/O constraints** — input/output delays for each interface, referenced to the right (real or virtual) clocks, from the interface specs and external device timing; drive/load. **(4) Add exceptions** — false paths for async crossings and static signals, multicycle paths where the design intends them (from the spec/designer knowledge). **(5) Environment/design rules** — operating conditions, max transition/cap. **(6) Verify coverage** — run `check_timing` to find unconstrained endpoints, missing clocks; iterate until coverage is complete; use `report_clocks` and query checks to confirm

application. The key discipline is *completeness and correctness verification* — because I didn't design it, I must confirm every clock is defined, every endpoint constrained, and every relationship/exception matches the real intent, using the tool's checks rather than assuming. I'd also engage the designer/spec for intent I can't infer (which paths are genuinely multicycle or false). Bringing up constraints from scratch is fundamentally about systematically covering all four categories and verifying nothing is left unconstrained or mis-specified.

Follow-up questions: 1. What's the first category of constraints to establish, and why? 2. How do you confirm you haven't left paths unconstrained?

Follow-up answers: 1. Clocks — because everything else (I/O delays, exceptions, all flop-to-flop timing) references them, so they must exist first; you define primary clocks at their sources with the spec frequencies, then generated clocks and relationships. 2. Run `check_timing` to report unconstrained endpoints, missing clocks, and undefined I/O delays, and iterate until it's clean; complement with `report_clocks` and direct object-query checks to confirm each constraint applied to the intended objects — actively verifying coverage rather than assuming it.

Q19. What are the consequences of over-constraining a design?

Why the interviewer asks: Tests understanding that too-tight constraints have costs.

Expected answer: Over-constraining — making the constraints tighter than the real requirement (too-short a period, padded uncertainty, unnecessary max-delay, overly pessimistic I/O delays) — has real costs even though it's "safe" in the sense of not hiding failures. Consequences: **(1) Wasted area and power** — synthesis and optimization work harder than needed, using larger, faster, more power-hungry cells and more buffering to meet an inflated target, increasing area and power for no functional benefit. **(2) Longer runtime and harder closure** — the tools spend effort closing paths that don't really need it, and closure takes longer or may not converge, delaying the schedule. **(3) Masking real priorities** — effort spent on artificially-critical paths can distract from genuinely critical ones. **(4) Potential for chasing false violations** — over-tight constraints produce violations that aren't real, consuming engineering time. So over-constraining isn't free: it trades area, power, runtime, and schedule for margin that isn't needed. The discipline (the margin-ledger mindset) is to constrain to the *real* requirement — enough margin for genuine uncertainty, not arbitrary tightening. Under-constraining is dangerous (hides failures); over-constraining is wasteful (costs area/power/schedule). Good constraint engineering targets the real requirement, avoiding both — which is why padding "to be safe" is itself a mistake, just a less catastrophic one than leaving things unconstrained.

Follow-up questions: 1. Name two concrete costs of over-constraining. 2. How does over-constraining compare to under-constraining in risk?

Follow-up answers: 1. Wasted area and power (larger, faster, more power-hungry cells and extra buffering to meet an inflated target) and harder/longer timing closure with increased runtime (tools work to close paths that don't really need it, delaying schedule); it can also cause chasing of false violations. 2. Under-constraining is *dangerous* — it hides

real failures (paths unchecked or under-checked, so silicon fails); over-constraining is *wasteful but safe* — it doesn't hide failures but costs area, power, runtime, and schedule for unneeded margin. Good practice avoids both by constraining to the real requirement.

Q20. How do you validate that a full SDC is complete and correct before signoff?

Why the interviewer asks: Tests the pre-signoff constraint validation methodology.

Expected answer: I'd validate across the four categories plus coverage and consistency, using tool checks rather than assumption. **(1) Coverage** — `check_timing` to confirm no unconstrained endpoints, no missing clocks, no undefined I/O; every sequential element reached by a clock; every port with appropriate I/O delay. **(2) Clocks** — `report_clocks` to verify every clock's period matches its target frequency and waveform, generated clocks have correct relationships, and async/exclusive groups are declared. **(3) I/O** — confirm input/output delays (both max and min facets) on all boundary ports, referenced to correct clocks, with realistic drive/load. **(4) Exceptions** — review every false/multicycle/max-min-delay: is each justified and correctly scoped (not over-broad, not hiding real paths)? Exceptions are audited especially carefully because a wrong one silently hides violations. **(5) Environment** — correct operating conditions/corners, design-rule limits. **(6) Consistency** — the SDC matches across synthesis/P&R/signoff (single source), and hierarchical block/top assumptions agree. **(7) Sanity on results** — do the reported critical paths, clock relationships, and exception counts make sense, or are there surprises indicating a constraint error? The mindset is that completeness (nothing unconstrained) and correctness (everything matches real intent, especially exceptions) must be *verified*, because the dangerous errors are silent. A signoff-quality SDC review actively confirms all four categories are complete and correct, exceptions are all justified, and the constraints are consistent across stages — treating a clean timing report as necessary but not sufficient, and the constraint review itself as the real assurance.

Worked example:

Pre-signoff SDC checklist:

```
COVERAGE   : check_timing clean (no unconstrained endpoints / missing clocks / no I/O)
CLOCKS      : report_clocks -- periods = target; gen-clocks correct; async/excl declared
I/O         : all ports have input/output delay (max AND min), right clock, real drive/load
EXCEPTIONS  : every false/MCP/max-min justified and correctly scoped (audit for hidden paths)
ENVIRONMENT: correct corners/operating conditions; max_transition / max_cap set
CONSISTENCY: same SDC across synth/PnR/signoff; block/top assumptions agree
SANITY      : critical paths, clock relations, exception counts make sense (no surprises)
```

Follow-up questions: 1. Why are exceptions audited especially carefully in a pre-signoff review? 2. What does `check_timing` contribute to SDC validation?

Follow-up answers: 1. Because a wrong or over-broad exception (false path, multicycle) silently *hides* real violations — it tells the tool not to check (or to relax) paths that may actually need checking, and since no violation appears, the missed problem is invisible; so each exception must be confirmed justified and correctly scoped. 2. It reports constraint

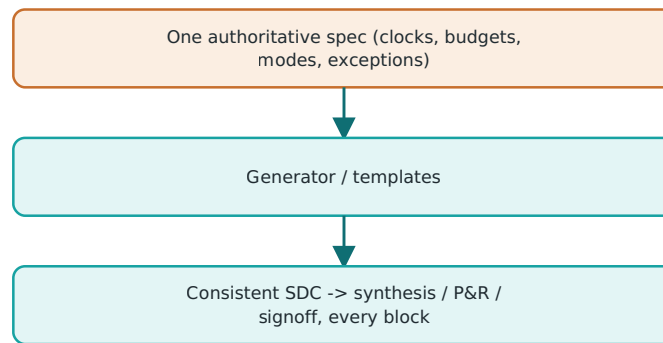
coverage gaps — unconstrained timing endpoints, missing clocks, undefined I/O delays — surfacing paths that aren't being checked (which wouldn't show as violations); it's the primary tool for confirming completeness, that nothing is silently left unconstrained before signoff.

Level 3 — Advanced (5-8 Years)

Q1. How would you architect a constraint methodology for a large SoC to ensure consistency across all stages and blocks?

Why the interviewer asks: Tests the ability to design a scalable constraint methodology, not just write SDC.

Expected answer: I'd build around a *single source of truth* and generation. **(1) One authoritative specification** of the timing intent — clock frequencies and relationships, interface budgets, modes, exceptions — from which all SDC is derived, so blocks and top and every stage agree by construction. **(2) Generated, templated constraints** — SDC produced by scripts from that spec rather than hand-written per block/stage, eliminating transcription drift and guaranteeing that a change propagates everywhere consistently. **(3) A layered structure** — common/shared constraints (clocks, global exceptions) defined once and included; block-specific constraints layered on; stage-specific refinements (ideal vs propagated, wire-load vs extracted) applied by stage. **(4) Explicit interface contracts** — each block's boundary assumptions (clock arrival, I/O budgets) specified centrally and checked against what the integrating context delivers, preserving hierarchical consistency. **(5) Automated verification** — `check_timing`, coverage checks, relationship-completeness checks, and cross-stage consistency checks run as gates on every block and at top. **(6) Version control and change governance** — the spec and generators under revision control, with change review (a constraint change ripples widely). The unifying principle is that constraints are *defined once, correctly, and propagated consistently* to all blocks and stages via generation and verification — because at SoC scale the dominant failure mode is *inconsistency* (blocks disagreeing with top, stages using different SDC), which a single-source, generated, verified methodology structurally prevents. Hand-maintained per-block/per-stage SDC files inevitably diverge; generation from one spec is what makes consistency achievable.

Single source of truth: generate consistent SDC for every block and stage

Follow-up questions: 1. Why is a single source of truth central to SoC constraint methodology? 2. What failure mode does generation-from-spec primarily prevent?

Follow-up answers: 1. Because at SoC scale, blocks, top, and every stage must agree on clocks, relationships, and interface budgets; deriving all SDC from one authoritative spec makes them consistent by construction, whereas independently hand-maintained constraints inevitably drift and disagree, causing integration and cross-stage failures. 2. Inconsistency — different blocks or stages using different periods, relationships, exceptions, or interface assumptions for the same design — which silently mis-times paths and invalidates block closure at integration; generating all SDC from one spec makes such divergence structurally impossible.

Q2. Explain how exceptions interact and take priority when multiple could apply to a path.

Why the interviewer asks: Tests deep knowledge of exception precedence — a subtle correctness issue.

Expected answer: When multiple exceptions could apply to a path, the tool resolves them by a defined *priority* order, and understanding it is essential to predicting which exception actually governs. The general precedence (highest to lowest): **false path** (`set_false_path`) is highest — if a path is false, that wins over everything (it's not timed at all). Next, **min/max delay** (`set_max_delay` / `set_min_delay`). Then **multicycle path** (`set_multicycle_path`). Lowest is the default clock-derived timing. Additionally, within the same exception type, a *more specific* specification generally takes priority over a more general one — an exception with more `-from` / `-through` / `-to` specificity (or referencing pins over clocks) typically overrides a broader one. So if a path is covered by both a false path and a multicycle, the false path wins (not timed). If covered by a max_delay and a multicycle, max_delay wins. This matters because in a large SDC, multiple exceptions can inadvertently overlap on a path, and the *effective* exception is determined by this priority — an engineer expecting a multicycle to apply might find a broader false path is actually suppressing it (or vice versa). Debugging "why isn't my exception taking effect?" often comes down to a higher-priority

exception overriding it. So knowing the precedence (false > max/min delay > multicycle > default, with specificity as a tiebreaker) lets you predict and control the effective constraint on overlapping paths.

Follow-up questions: 1. Which has higher priority: a false path or a multicycle path? 2. Within the same exception type, what breaks the tie between two applicable specifications?

Follow-up answers: 1. A false path — it has the highest exception priority, so if a path is covered by both a false path and a multicycle path, the false path wins and the path isn't timed at all (the multicycle is irrelevant because the path is excluded entirely). 2. Specificity — a more specific specification (more `-from` / `-through` / `-to` qualifiers, or referencing specific pins rather than whole clocks) generally overrides a more general one of the same type, so the tightest-scoped exception governs.

Q3. How do you constrain and verify a multi-mode design where modes share hardware but have different timing?

Why the interviewer asks: Tests multi-mode methodology depth.

Expected answer: In a multi-mode design (functional, test, low-power, different functional configurations), the same hardware is timed differently per mode — different active clocks, different I/O budgets, different exceptions, different paths active. The methodology: **(1) Define each mode as a scenario** — a set of constraints (clocks, case-analysis settings selecting the mode, I/O delays, exceptions) that describe that mode. Case analysis (`set_case_analysis` on mode-control signals) selects which logic/clocks are active. **(2) Analyze every mode** — the design must meet timing in *all* modes, because a path clean in one mode can fail in another (different clock, different active logic). **(3) Multi-corner-multi-mode (MMMC)** — combine each mode with each relevant corner, since the worst case is a specific mode×corner; the full matrix must close. **(4) Shared vs mode-specific constraints** — common constraints (base clocks) shared; mode-specific ones (which clock is active, mode exceptions) per scenario, ideally generated from one spec for consistency. **(5) Verify coverage per mode** — `check_timing` in each mode to ensure no mode leaves paths unconstrained. The essence is that a mode is a distinct timing view of shared hardware, so each must be separately and completely constrained and verified, and the design signs off only when the worst slack across the entire mode×corner matrix is acceptable. The risk is a mode being under-analyzed (e.g. test mode neglected) — a real, common escape, since a design fully closed in functional mode can still fail in a test or low-power mode that wasn't rigorously verified.

Follow-up questions: 1. Why must every mode be analyzed, not just the functional mode? 2. What selects which mode is active for a given analysis?

Follow-up answers: 1. Because the same hardware is timed differently per mode (different active clocks, active logic, I/O budgets, exceptions), so a path clean in functional mode can fail in test or low-power mode — the design must meet timing in *all* modes it operates in, and a neglected mode is a real escape route. 2. Case analysis (`set_case_analysis`) on the mode-control signals — setting those constants selects which

clocks and logic are active, so the analysis reflects that specific mode; each mode scenario has its own case-analysis settings defining it.

Q4. How would you debug a path that shows a violation you believe is false (not a real timing problem)?

Why the interviewer asks: Tests systematic debugging of suspected false violations.

Expected answer: A suspected-false violation usually means a *constraint* is wrong — the path is being timed when it shouldn't be, or timed against the wrong relationship. I'd investigate systematically: **(1)** `report_timing` on the path to see exactly how it's being timed — the launch and capture clocks, the edges, the path itself. **(2)** Ask: *is this a real functional path?* If launch and capture are in asynchronous domains, or the path can't functionally be exercised (mutually-exclusive modes, a false logical condition), then it *should* be excepted and isn't — the fix is a false path or clock-group/exclusivity declaration, or a case-analysis setting that reflects the real mode. **(3)** Check the clock relationship — is the tool pairing edges from clocks that are actually async or exclusive but weren't declared as such? That produces false violations from meaningless edge relationships. **(4)** Check for a missing multicycle — is the path functionally multicycle (result used every N cycles) but timed single-cycle? Then it needs a multicycle exception, not a false path. **(5)** Confirm with the designer/spec that the path is genuinely false/relaxed before excepting it — I don't except a path just to silence a violation; I confirm it's functionally not a real single-cycle path. The discipline is crucial: a false violation is fixed by adding the *correct, justified* exception (false, multicycle, clock-group, or case-analysis) that reflects the real functional intent — never by an unjustified exception that might also hide a real problem. So debugging a suspected-false violation is really determining the true functional nature of the path and encoding it correctly, with confirmation, rather than reflexively excepting it.

Follow-up questions: 1. What's the danger of excepting a path just to silence a violation? 2. How do you distinguish a false path from a missing multicycle path?

Follow-up answers: 1. You might hide a *real* timing problem — an unjustified exception tells the tool to stop checking a path that may actually need checking, so if the violation was real (or the exception is over-broad and catches other real paths), the problem becomes silent; exceptions must reflect confirmed functional intent, not just silence violations. 2. A false path is one that's *never* a real functional timing path (async crossing, mutually-exclusive modes, static signal) — it shouldn't be timed at all; a multicycle path *is* a real path but is functionally allowed *more than one cycle* (e.g. result used every N cycles) — it should be timed, just relaxed to N cycles. Determine whether the path functionally happens-but-slowly (multicycle) or never-really-happens-as-timed (false).

Q5. How do you approach budgeting I/O timing for a chip-to-chip interface where you own only one side?

Why the interviewer asks: Tests interface budgeting under partial control.

Expected answer: Owning one side of an interface means the total period budget must be split between our side and the external side, and we must budget our internal timing against the external side's parameters (from its datasheet or the interface spec). Approach: **(1) Get the interface budget** — the clock period and how it's allocated: for an *input* to us, the budget is $\text{period} = \text{external_clock_to_out} + \text{interconnect} + \text{our_setup} + \text{margin}$; for an *output* from us, $\text{period} = \text{our_clock_to_out} + \text{interconnect} + \text{external_setup} + \text{margin}$. **(2) Encode the external side as I/O delays** — `set_input_delay` = external clock-to-out + input interconnect; `set_output_delay` = external setup + output interconnect. These come from the external device's spec and the board interconnect estimate. **(3) Both facets** — max (setup) and min (hold) for each, since the external timing has a range. **(4) Reserve margin** for the uncertainties we don't control (external jitter, interconnect variation). **(5) Verify our internal paths meet the remaining budget** — what's left of the period after the external allocation. The key discipline is that we budget our internal side against a *conservative, spec-based* model of the external side, because we can't measure or control it — an optimistic assumption about the external device (too-small input delay, too-small external setup) directly risks an interface failure we can't see. So interface budgeting under partial control is about faithfully encoding the external side's worst-case timing (from its spec plus interconnect) as I/O delays, reserving margin for what we don't control, and closing our internal side against the remaining budget — treating the external parameters as a contract we must honor conservatively.

Follow-up questions: 1. Why must the external side's timing be modeled conservatively? 2. What determines how much of the period is available for our internal logic on an input path?

Follow-up answers: 1. Because we don't control or measure the external device — we rely on its datasheet/spec — so an optimistic assumption (too-small input delay or external setup) would make our internal budget look larger than it really is, risking an interface timing failure in silicon that we can't detect; conservative modeling of the external worst case protects against that. 2. The input delay — the period minus the external contribution (external clock-to-out + interconnect, encoded as `set_input_delay`) minus our capture flop's setup and margin leaves the budget available for our internal logic on that input path; a larger input delay leaves less internal budget.

Q6. What are the timing-methodology implications of physical (placement-aware) synthesis versus traditional synthesis?

Why the interviewer asks: Tests understanding of modern synthesis and constraint accuracy.

Expected answer: Traditional synthesis uses statistical *wire-load models* to estimate interconnect delay (no placement, so wire RC is guessed from fanout/block size), which becomes increasingly inaccurate at advanced nodes where interconnect dominates — leading to a large pre- vs post-layout timing gap (synthesis thinks paths are fast, real routing makes them slow). **Physical synthesis** integrates placement into synthesis, so it estimates interconnect from *actual placement-based* wire lengths rather than statistical models — far more accurate net delays, so the netlist synthesis produces is much closer to post-layout reality. Methodology implications: **(1)** The same SDC drives both, but physical synthesis's timing is more trustworthy, reducing surprises at P&R. **(2)** Constraint accuracy matters more upstream — with placement-aware timing, the constraints at synthesis directly shape a physically-realistic result, so getting them right (and consistent with signoff) at synthesis has more leverage. **(3)** The convergence between synthesis and signoff timing improves, so closure is more predictable. **(4)** It shifts effort earlier (more accurate early timing) rather than iterating post-layout. From a constraints standpoint, the commands are the same, but physical synthesis makes the *analysis those constraints drive* accurate early, so the constraints' influence on the final netlist is more direct and the pre/post-layout gap shrinks. The broad implication is that accurate early timing (physical synthesis) makes upstream constraint correctness more consequential and closure more predictable — reducing the traditional pattern of re-closing timing after layout reveals the real interconnect.

Follow-up questions: 1. Why does traditional synthesis have a large pre- vs post-layout timing gap? 2. How does physical synthesis change the leverage of upstream constraints?

Follow-up answers: 1. Because it uses statistical wire-load models to estimate interconnect (no placement, so wire RC is guessed) — at advanced nodes where interconnect dominates delay, these estimates are optimistic/inaccurate, so paths that look fast at synthesis become slow with real routing, creating a large gap requiring post-layout re-closure. 2. Physical synthesis estimates interconnect from real placement, so its timing is accurate early — meaning the constraints at synthesis shape a physically-realistic netlist directly; getting the upstream constraints right (and consistent with signoff) has more leverage on the final result, and closure becomes more predictable with fewer post-layout surprises.

Q7. How do you handle constraints across a hierarchical boundary with timing budgets?

Why the interviewer asks: Tests hierarchical budgeting methodology.

Expected answer: In a hierarchical flow, a top-level path may cross into a block, so the path's total budget must be *split* between top and block — this is timing budgeting. Approach: **(1) Allocate the budget** — for a path crossing the boundary, decide how much of the period the block's internal portion gets versus the top-level portion, and encode the block's share as I/O delays at the block boundary (input/output delays on the block ports representing the top-level portion of the path). **(2) The block closes** against those boundary budgets standalone. **(3) The top level** sees the block via a timing model (ETM/

ILM) carrying the block's internal timing, and closes the top portion. **(4) Consistency is essential** — the budget the block assumes at its boundary must match what the top level actually delivers/requires; if they disagree, the block closed against a wrong budget and integration fails. **(5) Budgeting can be iterative** — initial estimates refined as real timing emerges, keeping both sides consistent. Tools can automate budget derivation (deriving block I/O constraints from a top-level context). The core discipline is that the boundary budget is a *contract*: the block's assumed I/O timing and the top's delivered/required timing must agree, so the split is consistent and the sum (block portion + top portion) fits the period. Budgeting done well lets blocks close independently and integrate cleanly; done inconsistently (block and top assuming different splits), it produces integration timing failures that are expensive to debug. So hierarchical budgeting is fundamentally about a consistent, agreed split of the path budget across the boundary, encoded as matching I/O constraints on both sides.

Follow-up questions: 1. How is the block's share of a cross-boundary path's budget encoded? 2. Why must the block's boundary assumptions match the top level's?

Follow-up answers: 1. As I/O delays (input/output delays) on the block's boundary ports representing the top-level portion of the path — so the block, closing standalone, reserves the top's share and closes its internal portion against the remaining budget. 2. Because the block closes its internal timing against its assumed boundary budget; if the top level actually delivers/requires something different, the block was closed against a wrong budget, so it may fail (or be over-designed) when integrated — the boundary budget is a contract both sides must honor consistently for integration to work.

Q8. How would you validate that timing exceptions in a large SDC are all still valid and none are hiding real problems?

Why the interviewer asks: Tests auditing exceptions — a high-risk area.

Expected answer: Exceptions are high-risk because a wrong or stale one silently hides real violations, so I'd audit them systematically for *justification*, *scope*, and *currency*. **(1) Inventory every exception** — list all false paths, multicycle paths, and max/min delays (tools can report applied exceptions). **(2) Justify each** — for every exception, confirm the functional reason it exists (this crossing is async; this path is genuinely multicycle; this signal is static) and that the reason is still true. An exception without a documented, current justification is suspect. **(3) Check scope** — is each exception correctly and *narrowly* scoped (right `-from / -through / -to`)? An over-broad exception catches more paths than intended, potentially excepting real paths. This is the key risk — an exception meant for one path silently disabling checking on others. **(4) Check for staleness** — did a design change make an exception no longer valid (a path it excepts is now real, or a referenced object no longer exists)? Stale exceptions from old revisions are common. **(5) Cross-check against `check_timing` / analysis** — see if removing questionable exceptions reveals real violations that were being hidden. **(6) Confirm ignored/invalid exceptions** — the tool may report exceptions that didn't apply (object not found), indicating an error. The discipline is that every exception is a *deliberate suppression of checking* that must be

justified and scoped correctly, and the audit actively confirms each is valid, narrowly scoped, and current — because the failure mode (an exception hiding a real violation) is silent. Unjustified, over-broad, or stale exceptions are exactly where real problems hide, so exception audit is one of the highest-value constraint reviews.

Follow-up questions: 1. What's the primary risk of an over-broad exception? 2. Why are stale exceptions from previous design revisions dangerous?

Follow-up answers: 1. It excepts more paths than intended — an exception meant for one path (or set) silently disables or relaxes checking on other, real paths that happen to fall within its broad scope, hiding genuine violations with no symptom; correct, narrow scoping (precise `-from/-through/-to`) is essential. 2. Because a design change can make a previously-valid exception wrong — a path it excepts may now be a real functional timing path, or the functional condition that justified it may no longer hold — so the stale exception continues to suppress checking on a path that now needs it, silently hiding a real problem; exceptions must be re-validated against the current design.

Q9. How do you correlate synthesis/pre-layout timing predictions against post-layout signoff, and act on discrepancies?

Why the interviewer asks: Tests the pre/post correlation methodology.

Expected answer: The goal is that pre-layout (synthesis) timing predicts post-layout (signoff) well, so closure at synthesis means something. I'd correlate and act: **(1) Compare** the pre-layout critical paths and slacks against post-layout signoff for the same paths — how much did slack degrade, and where? **(2) Diagnose systematic gaps** — if post-layout is systematically worse, the pre-layout interconnect estimate (wire-load model) was optimistic; if specific paths degraded, their real routing was worse than estimated (long/congested nets). **(3) Act on the cause** — improve the pre-layout estimate (move to physical synthesis / placement-aware timing), or reserve margin at synthesis to cover the known gap, or fix the specific nets (better placement, buffering). **(4) Check constraint consistency** — a discrepancy can also come from *different constraints* pre vs post (an exception or clock difference), not just interconnect — rule that out by confirming the SDC matched. **(5) Feed back** — use the correlation to calibrate the pre-layout methodology so future predictions are better. The key judgment is distinguishing a *methodology* gap (optimistic interconnect estimate — fix by better estimation or margin) from a *specific-path* problem (a particular net routed badly — fix that net) from a *constraint inconsistency* (different SDC — fix the consistency). Good correlation makes synthesis timing trustworthy; poor correlation with unexamined causes means synthesis closure is illusory. So the methodology is to quantify the pre/post gap, attribute it (interconnect estimate, specific routing, or constraint mismatch), and act on the actual cause — closing the loop so early timing predicts final timing, which is what makes upstream closure meaningful.

Follow-up questions: 1. What are the likely causes if post-layout timing is systematically worse than pre-layout? 2. Besides interconnect, what else can cause a pre/post timing discrepancy?

Follow-up answers: 1. An optimistic pre-layout interconnect estimate — traditional wire-load models underestimate real wire RC (especially at advanced nodes where interconnect dominates), so paths that looked fast at synthesis degrade with real routing; the fix is placement-aware (physical) synthesis or reserving margin for the known gap. 2. Constraint inconsistency — different SDC pre vs post (a clock, exception, or I/O difference) times the same path differently, producing a discrepancy unrelated to interconnect; this must be ruled out by confirming the constraints matched across stages before attributing the gap to physical effects.

Q10. How do you constrain a design with clock domain crossings that use combinational logic (not just synchronizers) across the boundary?

Why the interviewer asks: Tests nuanced CDC constraining beyond simple flop synchronizers.

Expected answer: Some CDC structures involve combinational logic in the crossing (e.g. a mux-based data path selected by a synchronized enable, or logic between the launching domain and the synchronizer). The constraints must stop timing the async crossing while still bounding the physical delay where it matters. Approach: **(1) Declare the clocks asynchronous** (`set_clock_groups -asynchronous`) so the tool doesn't time the meaningless synchronous relationship across the boundary. **(2)** But async declaration disables *all* timing on the crossing, including bounding the combinational delay — which can matter: if multiple bits cross and must arrive within a bounded skew (so the receiving synchronizer sees consistent data), or if the combinational delay must be bounded for the crossing scheme to work, I'd apply a `set_max_delay` on the crossing combinational path to bound its delay (and possibly `set_min_delay`), even though the full synchronous check is disabled. **(3)** For a mux-based data crossing where the select is synchronized, ensure the select's synchronizer is timed (intra-domain) and the data mux delay is bounded. **(4)** The metastability/correctness of the crossing scheme is verified by CDC analysis, not STA. So the nuance is that async declaration alone can be *too* permissive — it disables all timing on the crossing, but some crossings need their combinational delay *bounded* (via `max_delay`) for correctness (bit-to-bit skew, settling), so a targeted delay constraint is added on top of the async declaration. This is more sophisticated than the simple "declare async and forget it" — it recognizes that the crossing has physical timing requirements (bounded skew/delay) even though the synchronous check is meaningless, and constrains those explicitly while relying on CDC for the metastability safety.

Follow-up questions: 1. Why might async declaration alone be insufficient for a combinational crossing? 2. What constraint bounds the crossing delay when the synchronous check is disabled?

Follow-up answers: 1. Because declaring the clocks async disables *all* timing on the crossing, but some crossings have real physical requirements — e.g. multiple bits must arrive within a bounded skew so the receiving synchronizer captures consistent data, or the combinational delay must be bounded for the scheme to work — which the blanket async declaration leaves unconstrained. 2. `set_max_delay` (and possibly `set_min_delay`)

applied to the crossing combinational path — it bounds the delay/skew of the crossing explicitly, providing the needed physical timing control even though the full synchronous setup/hold check is disabled by the `async` declaration.

Q11. How would you set up constraints to catch timing problems that only appear in specific corner/mode combinations?

Why the interviewer asks: Tests MMMC coverage strategy.

Expected answer: Some timing problems appear only at a specific *combination* of mode and corner (e.g. a path that's fine in functional mode at the slow corner but fails in test mode at the fast corner, or a cross-domain path worst at one voltage). Catching them requires deliberate MMMC coverage. Approach: **(1) Enumerate the modes** (functional, test, low-power, DVFS points) and the **corners** (PVT combinations, RC corners) relevant to each. **(2) Determine which combinations are meaningful** — not every mode runs at every corner, but every *realistic* mode×corner the silicon can experience must be covered. **(3) Analyze the matrix** — run STA across the mode×corner combinations, because the worst case for a given check can be a non-obvious combination (a mode's specific active paths interacting with a corner's specific delays). **(4) Use scenario definitions** — each mode×corner is a scenario with its clocks, case-analysis, and corner libraries; tools manage the matrix and report worst slack across it. **(5) Prune verified-dominated combinations** carefully — only skip a combination if you can *show* another dominates it for all its checks (inversion-aware: setup dominated by slow corners, hold by fast, but mode changes which paths are active). The key insight is that the worst case is often a *combination*, not any single mode or corner alone, so coverage must span the realistic matrix rather than analyzing modes and corners independently. The risk is analyzing each mode only at "its" corner and missing a cross-combination — a real escape route. So the strategy is deliberate, justified matrix coverage: every realistic mode×corner analyzed (or provably dominated), because the problems that hide in specific combinations are exactly the ones a mode-by-mode or corner-by-corner analysis misses.

Follow-up questions: 1. Why can the worst case be a mode×corner combination rather than any single one? 2. When is it safe to skip a mode×corner combination?

Follow-up answers: 1. Because a mode determines which paths are active and how they're timed (clocks, exceptions), while a corner determines the delays — a specific mode's active paths can be stressed by a specific corner's delays in a way that neither the mode at its "usual" corner nor the corner in another mode reveals, so the worst case emerges only from the combination. 2. Only when you can *demonstrate* it's dominated — another analyzed combination is provably worse for *all* of the skipped combination's checks (accounting for which paths the mode activates and inversion-awareness: setup worst at slow, hold at fast); skipping without proving dominance risks missing a problem that hides specifically in that combination.

Q12. How do you manage constraint complexity to keep a large SDC maintainable and reviewable?

Why the interviewer asks: Tests constraint software-engineering discipline.

Expected answer: A large SDC (thousands of lines, many exceptions) becomes unmaintainable and un-reviewable without discipline, so I'd apply software-engineering practices. **(1) Structure and modularity** — organize by category (clocks, I/O, exceptions, environment) and by block, in clearly-named, commented files, so a reviewer can navigate. **(2) Generate from a spec** — derive repetitive/parameterized constraints (bus I/O delays, per-domain clocks) via scripts from a single source, reducing hand-written volume and error, and making intent (the spec) reviewable rather than thousands of literal commands. **(3) Comment the why** — especially exceptions: every false/multicycle path should have a comment stating its functional justification, so a reviewer (or future engineer) can validate it without archaeology. **(4) Minimize and scope exceptions** — fewer, narrowly-scoped exceptions are more maintainable and less risky than many broad ones; prefer clock groups over pairwise false paths. **(5) Version control and change review** — track changes, review constraint diffs (a constraint change can ripple widely). **(6) Automated checks** — `check_timing`, coverage, and consistency checks catch regressions. **(7) Documentation** — the constraint methodology and interface contracts documented. The goal is that the SDC is *understandable and verifiable*: a reviewer can see the structure, understand each exception's justification, trust that repetitive constraints are generated consistently, and catch regressions automatically. Unstructured, uncommented, hand-written SDC with unexplained exceptions is where errors accumulate and hide; treating constraints as maintainable code (structured, generated, commented, version-controlled, checked) keeps a large SDC correct and reviewable as it grows. This matters because constraint errors are silent and consequential, so maintainability directly serves correctness.

Follow-up questions: 1. Why should every exception carry a comment stating its justification? 2. How does generating constraints from a spec aid maintainability?

Follow-up answers: 1. Because an exception silently suppresses checking, so its validity must be reviewable — a comment stating the functional justification (why this path is false/multicycle) lets a reviewer or future engineer confirm it's still valid without reverse-engineering the intent, and flags exceptions that lack a defensible reason (which are suspect). 2. It reduces hand-written volume (less transcription error), guarantees repetitive constraints (bus I/O, per-domain clocks) are consistent, and makes the *spec* the reviewable artifact rather than thousands of literal commands — so intent is clear, changes propagate consistently, and the SDC stays correct and understandable as it scales.

Q13. How would you constrain and analyze retention/isolation and power-domain-related timing in a low-power design?

Why the interviewer asks: Tests awareness of low-power (UPF) timing interactions.

Expected answer: Low-power designs use power domains that can be shut off, with special cells — isolation cells (clamp outputs of a powered-down domain to a safe value) and retention flops (preserve state through power-down) — described in a power-intent format (UPF) alongside the SDC. The timing implications the constraints/analysis must handle: **(1) Isolation cells** — the isolation enable signal has timing (it must assert before power-down and de-assert after power-up, with setup/hold relative to the relevant control/clock), so isolation control paths are timed. **(2) Retention flops** — the save/restore control signals have timing requirements (save must complete before power-down; restore relative to power-up), which must be constrained and checked. **(3) Power-domain crossings** — signals crossing between domains (especially between an always-on domain and a switchable one) have their own timing, and the always-on control logic timing matters. **(4) Multiple power states** — like modes, each power state (which domains on/off, at what voltage) is a scenario to analyze, since timing differs by state (a domain at a lower retention voltage, or the interactions during power transitions). **(5) Level shifters** between voltage domains have timing (added delay). The analysis combines the power intent (UPF: which domains, isolation/retention strategy) with timing (the control-signal timing, the per-state analysis). So constraining low-power timing is about timing the *control* signals (isolation enable, retention save/restore) correctly, analyzing each power *state* as a scenario, and accounting for the special cells' (isolation, retention, level-shifter) timing — an interaction between power intent and timing that a purely functional-mode STA would miss. The risk is neglecting the power-control timing or a specific power state, letting a power-transition or state-specific timing bug escape.

Follow-up questions: 1. What timing do isolation and retention cells introduce that must be constrained? 2. Why is each power state analyzed as a separate scenario?

Follow-up answers: 1. The control-signal timing — isolation enable must assert/de-assert with proper setup/hold relative to power-down/up and the relevant clock/control; retention save/restore signals must meet their timing (save complete before power-down, restore relative to power-up) — so these control paths, plus level-shifter delays, must be constrained and checked. 2. Because timing differs by power state — which domains are on/off, at what voltage (e.g. a retention voltage), and the interactions during transitions — so a path or control signal fine in one state can fail in another; like modes, each realistic power state must be analyzed to catch state-specific timing problems.

Q14. How do you use constraints to guide and verify a timing ECO (engineering change order)?

Why the interviewer asks: Tests the ECO timing-fix methodology.

Expected answer: A timing ECO makes targeted netlist changes (resize cells, add buffers, restructure a path) late in the flow to fix remaining violations without disrupting the whole design. Constraints' role: **(1) The same signoff SDC** drives the ECO analysis — the ECO is verified against the authoritative constraints, so the fix is checked against the real requirements. **(2) Identify the violating paths** precisely (the constraints define which paths violate and by how much — WNS/TNS), so the ECO targets exactly them. **(3)**

Incremental analysis — after the ECO change, re-time incrementally to confirm the targeted paths now meet timing *and* that the change didn't break neighbors (an ECO fix can create a new violation elsewhere — e.g. fixing setup by upsizing can hurt a hold path, or adding delay for hold can hurt setup). The constraints define both checks, so both must be re-verified. **(4) Verify no new violations** — the whole (or affected region's) timing re-checked against the full SDC to ensure the ECO didn't introduce a regression. **(5) Preserve exceptions/relationships** — the ECO must respect the existing exceptions and clock relationships (not, e.g., create a path that should be excepted but isn't). The discipline is that an ECO is a *local* change verified against the *full* constraints — the fix must resolve the target violation without violating any other constraint (setup, hold, design rules) that the SDC defines. The risk is a "fix" that closes the target but silently opens another violation (the setup/hold tension especially), so the ECO methodology relies on the constraints to verify both the fix and the absence of new problems. So constraints guide the ECO by defining the target precisely and verifying comprehensively that the local fix is globally clean.

Follow-up questions: 1. Why must an ECO be re-verified against both setup and hold? 2. What's the risk of a timing ECO that only checks the targeted path?

Follow-up answers: 1. Because the setup/hold tension means a fix for one can break the other — upsizing a cell or adding delay to fix setup can create a hold violation (or vice versa); the constraints define both checks, so an ECO must confirm the fix resolves the target *and* doesn't violate the opposite check on that path or its neighbors. 2. It may close the targeted violation while silently opening a new one elsewhere — the local change (added buffer, resized cell) can degrade neighboring paths or the opposite check, and if only the target is re-checked, that new violation escapes; the ECO must be verified against the full constraints over the affected region, not just the target path.

Q15. How do you correlate the timing signed off with constraints against actual silicon behavior?

Why the interviewer asks: Tests the silicon-correlation loop for constraints.

Expected answer: Silicon correlation validates that the constraint-driven signoff actually bounds real behavior — that the design works at the frequencies and conditions the constraints claimed. Methods and actions: **(1) At-speed testing** — run the silicon at the target frequency (and margins) the constraints signed off for; passing confirms the setup timing (period budget) was adequate, failing indicates optimism. **(2) Shmoo plots** — characterize the pass/fail boundary across frequency and voltage, comparing where silicon actually fails against where the constrained analysis predicted the margin runs out; a silicon failure at a frequency the constraints said should pass indicates the analysis (or a constraint) was optimistic. **(3) Path-specific measurement** — where possible, measure specific critical paths' delays against predictions. **(4) Investigate discrepancies** — if silicon fails where analysis passed, determine whether a constraint was wrong (optimistic period, missing margin, a wrongly-excepted path that's actually real and slow), a model was optimistic, or a specific path was mis-analyzed. The *dangerous* direction is silicon

worse than signoff (optimism → escapes); silicon better means recoverable margin. **(5) Feed back** — correct the constraints/margins so future signoff bounds reality. The correlation specifically tests the constraints' claims: the period (does it run at that frequency?), the exceptions (is a supposedly-false path actually causing a problem?), the margins (was the uncertainty enough?). A constraint-level correlation failure — e.g. a path excepted as false that actually violates in silicon — is a serious finding, because it means checking was wrongly suppressed. So silicon correlation closes the loop on constraints: it confirms the constraint-driven signoff's central claim (the design meets timing at the specified conditions) against physical reality, and any discrepancy is traced to and corrected in the constraints, models, or margins — because the constraints are ultimately a *claim about silicon* that must be validated.

Follow-up questions: 1. What does at-speed silicon testing validate about the constraints? 2. Why is a false-path exception that actually violates in silicon a serious finding?

Follow-up answers: 1. That the setup timing budget — the clock period the constraints signed off for — was adequate: running the silicon at the target frequency and passing confirms the paths really do meet that period; failing indicates the period budget (or margin) was optimistic, i.e. the constraints claimed a frequency the silicon can't achieve. 2. Because it means checking was *wrongly suppressed* — a path declared false (not timed) is actually a real functional path that's too slow, so the constraint hid a genuine violation that then appears in silicon; it reveals an incorrect exception that made the signoff claim a timing correctness it didn't actually verify, a direct optimism/escape.

Q16. How would you constrain a design to allow safe clock-domain crossing while still bounding the crossing's physical timing?

Why the interviewer asks: Tests the balance of async declaration and physical bounding (deepens L2 Q14/L3 Q10).

Expected answer: The two requirements — don't time the meaningless synchronous relationship, but do bound the crossing's physical timing where correctness needs it — are met by combining async declaration with targeted delay constraints. **(1) Declare async** (`set_clock_groups -asynchronous`) so STA doesn't time the crossing synchronously (no meaningless setup/hold against unrelated edges). **(2) Bound where needed** — for crossings whose correctness depends on physical timing, add explicit bounds: for a multi-bit crossing captured by a synchronizer, the bits must arrive within a bounded relative skew (so the receiver samples a consistent value or the gray-code property holds), enforced by `set_max_delay` on the crossing nets (and matching them); for a single-bit crossing into a synchronizer, `set_max_delay` can bound the combinational delay so the data is stable when sampled. **(3) The synchronizer's intra-domain timing** is checked normally (it's in the receiving clock domain). **(4) CDC analysis** verifies the structural correctness and metastability settling. So the crossing is *functionally* made safe by the synchronizer design (verified by CDC), *not-mis-timed* by the async declaration, and *physically bounded* where needed by targeted max_delay constraints. The sophistication is recognizing these are three distinct concerns: metastability safety (CDC + synchronizer design), suppressing the

meaningless synchronous check (async clock groups), and bounding real physical timing like bit-skew (`max_delay`). A naive approach does only the async declaration and misses the physical bounding, which can let a multi-bit crossing capture inconsistent data despite "passing" — a real, subtle failure. So the complete constraint approach addresses all three, with the `max_delay` bounds being the piece that a simplistic treatment omits.

Follow-up questions: 1. What are the three distinct concerns for a CDC in this approach? 2. What can go wrong if only the async declaration is done for a multi-bit crossing?

Follow-up answers: 1. Metastability safety (handled by the synchronizer design and verified by CDC analysis), suppressing the meaningless synchronous timing check (handled by `set_clock_groups -asynchronous`), and bounding the crossing's real physical timing like bit-to-bit skew (handled by targeted `set_max_delay` constraints) — three separate concerns requiring three distinct treatments. 2. The multi-bit crossing's bits can arrive with unbounded relative skew, so the receiving synchronizer may capture an *inconsistent* combination of bits (some old, some new) — breaking schemes that rely on bounded skew (e.g. gray-code pointers) — a real, subtle failure that "passes" STA (async, not timed) but malfunctions in silicon because the physical skew wasn't bounded by a `max_delay`.

Q17. How do you decide the appropriate margin (uncertainty, derates, guardbands) to build into constraints?

Why the interviewer asks: Tests principled margin-setting across the constraint set.

Expected answer: Margin should equal the sum of the *real, justified, non-redundant* uncertainties the design faces — the margin-ledger discipline — not an arbitrary pad. I'd account for each real source once, with the appropriate mechanism: **(1) Clock uncertainty** — jitter (from the clock source spec), skew (estimated pre-CTS, real in latency post-CTS so removed from uncertainty then), and a defensible safety margin. **(2) OCV/derates** — on-chip variation, sized to the real device variation (flat derate, or better, POCV/AOCV statistical modeling placing margin where variation actually is). **(3) Model guardband** — margin for delay-model inaccuracy, sized to the known model error. **(4) Aging** — if the product lifetime warrants it (more for automotive). **(5) IR-drop** — margin for supply droop if not otherwise analyzed. The key disciplines: *justify each* (tie it to a real physical effect and its magnitude), *don't double-count* (the same effect covered by a derate and an uncertainty and a corner is redundant over-margin — e.g. skew in both uncertainty and propagated latency post-CTS), and *validate against silicon* (correlation confirms the total margin bounds reality). Too little margin risks escapes; too much wastes power/area/schedule. So the decision is contributor-by-contributor: enumerate the real uncertainties, cover each once with the right mechanism, size it to the real (worst-case) magnitude, avoid redundancy, and validate the total against silicon. Margin isn't a single comfort number; it's a *ledger* of justified, non-overlapping contributions — which is what keeps the design neither under-margined (dangerous) nor over-margined (wasteful).

Follow-up questions: 1. What is the risk of double-counting a margin contributor? 2. How should the margin for on-chip variation ideally be applied?

Follow-up answers: 1. Over-margin — the same physical effect (e.g. clock skew) covered by two mechanisms at once (uncertainty *and* propagated latency, or a derate *and* a corner) adds margin twice for one real uncertainty, over-constraining the design and wasting power/area/schedule for protection that's already provided; each real effect should be covered exactly once. 2. Statistically (POCV/AOCV) rather than as a flat blanket derate — modeling each arc's variation and combining them by root-sum-square places margin where the variation actually is and credits that independent variations don't all peak together ($\sqrt{N}$, not N), giving less pessimism placed more accurately than a uniform derate that assumes every stage is simultaneously worst-case.

Q18. How would you approach constraints for a design that must support multiple bin frequencies (speed grades)?

Why the interviewer asks: Tests multi-frequency signoff (speed-binning) methodology.

Expected answer: Speed-binned parts are sold at multiple frequency grades (a fast bin, a slow bin), so the design must be characterized and, in effect, signed off across the frequency range it will be binned into. Approach: **(1) Define the frequency range** — the set of target frequencies (bins) the part supports, from the fastest bin down. **(2) Setup at each frequency** — setup timing must be verified at each bin frequency (each is a different period budget); the fastest bin is the tightest setup, so it's the hardest, but each bin's frequency must close for parts sold at that bin. **(3) Hold across the range** — hold is largely period-independent (same-edge race), so it must hold at all frequencies, checked at the fast corner. **(4) Voltage/frequency pairing** — bins may pair frequency with voltage (a fast bin at higher voltage), so each bin is a (frequency, voltage) scenario. **(5) The design closes for a bin** if it meets setup at that bin's frequency and hold across all. Parts that meet the fastest bin's setup are sold as fast; parts that only meet a slower bin are sold slower (binning by measured capability). The constraint methodology treats each bin as a scenario (period + conditions) and verifies the design across the binning range — the fastest bin bounds setup design effort, hold must hold everywhere. The connection to DVFS is close (multiple frequency/voltage operating points), but binning is about *sorting parts by capability* rather than one part changing frequency at runtime. So constraints support binning by defining the range of bin frequencies as scenarios and verifying setup at each (fastest is hardest) and hold across all — enabling the design to be sold across speed grades with each grade's timing guaranteed.

Follow-up questions: 1. Which bin frequency bounds the setup design effort, and why? 2. Why must hold be verified across all bin frequencies?

Follow-up answers: 1. The fastest bin — it has the shortest period and thus the tightest setup budget, so it's the hardest setup target; designing to meet the fastest bin's setup covers the slower bins' setup (which have more time), so the fastest bin bounds the setup effort. 2. Because hold is a same-edge race that's largely independent of the clock period, so a hold violation isn't fixed by running slower — a part will be operated at various bin frequencies, and hold must hold at all of them (checked at the fast corner); a hold failure would affect the part at every speed grade.

Q19. How do you ensure timing exceptions written for one tool or stage translate correctly to another?

Why the interviewer asks: Tests cross-tool/stage exception portability.

Expected answer: Exceptions must mean the same thing across tools (synthesis, P&R, signoff, possibly from different vendors) and stages, or a path excepted in one but not another is timed inconsistently. Ensuring this: **(1) Use standard SDC** — stick to standard SDC exception commands and semantics that all tools interpret consistently, avoiding tool-specific extensions that don't translate. **(2) Consistent object references** — exceptions reference objects (pins, nets, cells) by names that must exist and match across the tools' netlist views; a name that resolves in one tool but not another (due to hierarchy or naming differences) makes the exception apply in one and silently not in the other. Naming consistency across the flow is essential. **(3) Verify application in each tool** — confirm (via the tool's exception reporting) that each exception actually applied to the intended paths *in each tool/stage*, not just where it was written — catching the silent "didn't match here" case. **(4) Single source, generated** — derive exceptions from one spec so the same intent flows to all tools, rather than re-writing per tool (which invites divergence). **(5) Watch for semantic differences** — some tools interpret exception priority or scoping subtly differently; validate that overlapping exceptions resolve the same way. The core risk is an exception that applies in one tool/stage and *silently doesn't* in another (usually a naming/matching issue) — so the path is excepted at signoff but timed at synthesis (or vice versa), causing inconsistency. The defense is standard semantics, consistent naming, single-source generation, and — crucially — verifying in *each* tool that the exception applied as intended. So exception portability is ensured by consistency (standard SDC, matching names, one source) plus per-tool verification of application, because the failure mode is silent non-application in one context.

Follow-up questions: 1. What's the main reason an exception might apply in one tool but not another? 2. How do you confirm an exception translated correctly to a given tool?

Follow-up answers: 1. Object-name mismatch — the exception references pins/nets/cells by names that resolve in one tool's netlist view but not another's (due to hierarchy separators, naming conventions, or optimization renaming), so the query matches the intended objects in one tool and silently matches nothing in the other, applying there to zero paths. 2. Use the tool's exception/timing reporting to confirm the exception actually applied to the intended paths *in that tool* — checking it covers the expected paths (non-empty, correct scope) rather than assuming; this catches the silent non-application case where the exception was written but didn't match in that tool's context.

Q20. How would you conduct a signoff-quality review of a complete SDC for a critical design?

Why the interviewer asks: Integrative advanced review capability.

Expected answer: A signoff-quality SDC review is systematic across all four categories plus coverage, consistency, and justification, using tool evidence rather than assumption — because constraint errors are silent and consequential. **(1) Coverage** — `check_timing` clean: no unconstrained endpoints, no missing clocks, no undefined I/O; every sequential element clocked; every port with I/O delays (both facets). **(2) Clocks** — every clock's period matches target frequency and waveform; generated clocks correct (source, factor, relationship); async/exclusive relationships all declared; latency/uncertainty stage-appropriate and justified. **(3) I/O** — input/output delays (max and min) on all boundary ports, right clocks, realistic drive/load; interface budgets consistent with the external side. **(4) Exceptions** — every false/multicycle/max-min-delay inventoried, each *justified* (documented functional reason, still valid), correctly and narrowly *scoped* (not over-broad), and not *stale*; this is the highest-risk area (exceptions silently hide violations), so audited most carefully. **(5) Environment** — correct corners/operating conditions covering the mode×corner matrix; design-rule limits set. **(6) Consistency** — same SDC (single source) across synthesis/P&R/signoff; hierarchical block/top assumptions agree; exceptions translate across tools. **(7) Margin** — uncertainty/derates justified, non-redundant, silicon-correlated. **(8) Sanity** — reported critical paths, clock relationships, and exception counts make sense; investigate surprises as possible constraint errors. The mindset: completeness (nothing unconstrained), correctness (everything matches real intent), and justification (every exception defensible) must be *actively verified* with tool evidence, treating a clean timing report as necessary but not sufficient. For a critical design I'd also have an independent reviewer, document the review, and correlate against silicon where possible. The review's purpose is to confirm the constraints — the specification the whole signoff rests on — are complete, correct, justified, and consistent, because that is what makes the timing signoff trustworthy.

Worked example:

Signoff-quality SDC review dimensions:

```

COVERAGE      : check_timing clean (no unconstrained EPs / missing clocks / missing I/O)
CLOCKS         : periods=target; waveforms right; gen-clocks correct; async/excl declared
I/O            : all ports input+output delay (max AND min); right clock; real drive/load
EXCEPTIONS     : every one inventoried, JUSTIFIED, narrowly SCOPED, not STALE (highest risk)
ENVIRONMENT    : mode x corner matrix covered; design-rule limits set
CONSISTENCY    : single-source SDC across synth/PnR/signoff; block/top agree; cross-tool ok
MARGIN         : uncertainty/derates justified, non-redundant, silicon-correlated
SANITY         : critical paths / clock relations / exception counts sensible (probe surprises)
PROCESS        : independent reviewer + documented review + silicon correlation where possible

```

Follow-up questions: 1. Which SDC area is the highest-risk and warrants the most careful review, and why? 2. Why is a clean timing report necessary but not sufficient for signoff-quality constraints?

Follow-up answers: 1. Timing exceptions — because each one silently *suppresses or relaxes checking* on paths, so a wrong, over-broad, or stale exception hides real violations with no symptom; they must each be inventoried, justified (valid functional reason), narrowly scoped, and confirmed current, since this is where real problems most easily hide undetected. 2. Because a clean report only means the paths the tool was *told* to check passed — it says nothing about paths left unconstrained (missing clocks/I/O) or wrongly excepted (over-broad false paths), which produce no violations precisely because they're

not being checked; so completeness, correctness, and exception justification must be actively verified, with the clean report being necessary but not proof of correctness.

Level 4 — Expert (8+ Years: Senior/Lead/Staff)

Q1. Design a complete constraint methodology and governance model for a multi-team SoC program. What are the pillars?

Why the interviewer asks: The staff-level question — can you own the constraint discipline across a whole program?

Expected answer: Pillars: **(1) A single source of constraint truth** — one authoritative specification of timing intent (clock frequencies and relationships, interface budgets, modes, exceptions with justifications) from which all SDC is generated, so every block and every stage agrees by construction rather than by manual coordination. **(2) Generation over hand-writing** — SDC produced by scripts/templates from that spec, eliminating transcription drift and guaranteeing a change propagates everywhere consistently. **(3) Layered, modular structure** — shared/common constraints defined once and included; block-specific layered on; stage-specific refinements (ideal vs propagated, wire-load vs extracted) applied by stage. **(4) Explicit interface contracts** — each block's boundary assumptions specified centrally and checked against what the integrating context delivers, preserving hierarchical consistency. **(5) Automated verification as gates** — `check_timing` coverage, relationship completeness, exception justification, and cross-stage consistency checks run on every block and at top, blocking progress on failure. **(6) Exception governance** — every exception requires a documented justification, is reviewed, and is periodically re-validated (exceptions are where real violations hide). **(7) Change control** — the spec and generators under version control with review, because a constraint change ripples widely. The unifying principle: constraints are *defined once, correctly, generated consistently, and verified automatically* across all blocks and stages — because at program scale the dominant failure mode is *inconsistency and unjustified exceptions*, which a single-source, generated, governed, gated methodology structurally prevents. Hand-maintained per-block/per-stage SDC inevitably diverges and accumulates stale exceptions; the methodology's job is to make correctness and consistency the default, not a heroic manual effort.

Follow-up questions: 1. Why is generation-from-spec superior to hand-written SDC at program scale? 2. Why does exception governance deserve its own pillar?

Follow-up answers: 1. Because generation guarantees consistency (all blocks/stages derive from one spec, so they can't drift), eliminates transcription errors, and makes a change propagate everywhere at once — whereas hand-written per-block/per-stage SDC inevitably diverges as the design and team scale, producing the inconsistencies that cause integration and cross-stage failures. 2. Because exceptions silently suppress checking, so an unjustified, over-broad, or stale exception hides real violations with no symptom — the single highest-risk constraint category; governing them (mandatory justification, review,

periodic re-validation) directly prevents the most dangerous, hardest-to-detect class of constraint error.

Q2. How would you decide the overall constraint architecture (flat vs hierarchical, budgeting strategy) for a large design?

Why the interviewer asks: Tests high-level methodology judgment.

Expected answer: The choice follows the design's size, team structure, and reuse needs.

Flat (whole design constrained and closed as one) is simplest and avoids budgeting/consistency issues, but doesn't scale — runtime, capacity, and team parallelism break down on a large SoC. **Hierarchical** (blocks constrained and closed independently, then integrated via timing models) scales and enables parallel teams and IP reuse, but introduces the budgeting and consistency burden — each block's boundary assumptions must match what the top delivers, or integration fails. I'd choose hierarchical for a large multi-team SoC, with: **(1)** a clear block partitioning aligned to teams and reuse; **(2)** a budgeting strategy allocating cross-boundary path budgets consistently (block I/O constraints derived from the top context, ideally generated); **(3)** timing models (ETM/ILM) representing blocks at top; **(4)** interface contracts checked for consistency; **(5)** possibly a hybrid — hierarchical for large reusable blocks, flat within a moderate block. The key judgment is that hierarchy buys scalability and parallelism at the cost of budgeting/consistency discipline, so it's warranted when the design is too large for flat closure or when parallel teams/reuse demand it — and when chosen, the budgeting and interface-contract consistency must be rigorously managed (the hierarchical flow's characteristic risk). For a moderate design where flat closure is feasible, flat is simpler and avoids those risks. So the architecture decision weighs scalability/parallelism (favoring hierarchy) against budgeting/consistency complexity (favoring flat), sized to the design.

Follow-up questions: 1. What does a hierarchical flow buy, and what does it cost? 2. What's the characteristic risk that must be managed in a hierarchical constraint flow?

Follow-up answers: 1. It buys scalability (large designs closeable in pieces), team parallelism (blocks closed independently), and IP reuse; it costs the budgeting and consistency burden — cross-boundary path budgets must be allocated and each block's boundary assumptions must match what the top delivers. 2. Boundary inconsistency — a block closes against assumed I/O budgets that don't match what the top level actually delivers/requires, so the block's closure is invalid at integration; managing it requires consistent (ideally generated) budgeting and checked interface contracts between block and top.

Q3. How do you architect constraints to be robust to late specification changes (frequency, interface, mode)?

Why the interviewer asks: Real programs change specs late; tests design-for-change.

Expected answer: Late changes (a frequency bump, an interface revision, a new mode) are inevitable, and hard-coded constraints make them error-prone. I'd architect for change:

(1) Parameterize — frequencies, interface budgets, and mode configurations expressed as variables/parameters in one spec, so a change updates one value that propagates. **(2) Derive by relationship** — generated clocks by ratio, related budgets by formula, so a master change correctly rescales dependents. **(3) Generate the SDC** from the parameterized spec, so a change is a spec edit + regenerate, not a manual hunt-and-edit across files. **(4) Modularize by concern** — clocks, I/O, exceptions, modes in separate generated sections, so an interface change touches only the I/O module, a mode change only the mode module. **(5) Regression verification** — automated `check_timing`, coverage, and consistency checks re-run after regeneration to confirm nothing broke. **(6) Version control** the spec so changes are tracked and reviewable. The principle: express constraints as *parameters and relationships* in a single generated source, so late changes are localized, propagated automatically, and verified — rather than requiring risky manual edits across many hand-written files. This is the difference between a constraint setup that absorbs late spec churn safely and one that breaks under it, introducing subtle inconsistencies exactly when schedule pressure is highest. Designing for change up front (parameterization, generation, modularity, regression checks) is what makes late changes low-risk.

Follow-up questions: 1. How does parameterization make a late frequency change low-risk? 2. Why regenerate rather than hand-edit the SDC for a spec change?

Follow-up answers: 1. The frequency is defined as one parameter in the spec, and dependent clocks/budgets are derived from it by relationship, so changing that one value updates everything consistently on regeneration — versus hand-editing many absolute values across files, where it's easy to miss one or introduce an inconsistency. 2. Because hand-editing many files for a spec change is error-prone (miss a constraint, mistype, create inconsistency), while regenerating from a single parameterized, modular spec updates everything consistently and is repeatable and reviewable — turning a risky manual sweep into a controlled parameter change plus automated regression verification.

Q4. Post-silicon, a design fails timing. How do you use the constraints to systematically investigate?

Why the interviewer asks: The postmortem, from the constraint angle.

Expected answer: I'd use the constraints as the map to systematically localize the failure, because they define what was (and wasn't) verified. **(1) Characterize the failure** — at what frequency/voltage/temperature does it fail, and is it a setup (fails fast, works slow) or hold (fails regardless of frequency) symptom? Setup points to the period budget; hold to a fast-path/skew issue. **(2) Check whether the failing behavior was even analyzed** — was there a constraint gap (a missing clock, an unconstrained path, an over-broad false path) that left the failing path unchecked? A silicon failure on a path the constraints never checked is a constraint-coverage escape. **(3) Check whether an exception wrongly suppressed it — is the failing path covered by a false path or multicycle that's actually invalid? A wrongly-expected real path is a classic escape.** **(4) Check**

whether a constraint was optimistic — too-small I/O delay, too-loose period, insufficient margin/uncertainty — so the path was analyzed but against a wrong (optimistic) requirement. **(5) Check corner/mode coverage** — did the failing condition (a specific mode×corner) get analyzed, or was it a coverage gap? **(6) Correlate** — compare the failing path's silicon behavior against its signed-off analysis to see where the model/constraint was optimistic. The systematic move is to treat the constraints as the specification of what was verified and ask, for the failing path: was it checked at all (coverage gap), checked but wrongly excepted (exception error), or checked against a wrong requirement (optimistic constraint)? Each points to a different constraint root cause and fix. This constraint-centric triage quickly localizes whether the escape is a coverage hole, a bad exception, or an optimistic constraint — the three constraint-level failure modes — before diving into models or physics.

Follow-up questions: 1. What are the three constraint-level root causes of a timing escape? 2. How does the setup-vs-hold symptom guide the investigation?

Follow-up answers: 1. A coverage gap (the failing path was never checked — missing clock, unconstrained endpoint, or a path excluded by an over-broad exception), a wrong exception (a false/multicycle exception invalidly suppressed a real path), or an optimistic constraint (the path was checked but against a too-loose requirement — small I/O delay, loose period, insufficient margin). 2. Setup ("fails fast, works slow") points to the period budget — an optimistic period, insufficient margin, or a slow path against the clock — so investigate the setup-side constraints and margins; hold (fails regardless of frequency) points to a fast-path/skew issue — a hold-side constraint, min-delay, or clock-skew problem — since hold is period-independent, narrowing where to look.

Q5. How should constraint rigor differ for a safety-critical (automotive/aerospace) versus a consumer design?

Why the interviewer asks: Differentiated rigor by consequence.

Expected answer: Same constraint mechanics, differentiated margin, coverage, evidence, and process. **Safety-critical (ISO 26262 / DO-254):** larger, justified margins (more conservative for the harsher, longer-life, wide-temperature environment and aging); verification across the full environmental and aged envelope (not just nominal corners); *traceable, documented* constraint correctness for certification (evidence that every clock is defined, every exception justified, every mode/corner covered, with audit trails); independent review of constraints and exceptions; and often fault-detection features (e.g. clock monitors) that themselves need constraining. Exceptions face especially strict scrutiny (an unjustified exception hiding a violation is unacceptable where safety is at stake). **Consumer:** the same methodology but margins sized to consumer conditions/lifetime, lighter documentation, faster iteration, and emphasis on not over-margining (excess margin wastes cost/power at volume). The distinctive safety-critical additions are the *evidence and traceability* (you must *prove* the constraints are complete and correct for certification, not just believe it) and the *environmental/aging coverage* (the design must be shown robust across a demanding, long-life envelope). The principle (differentiated rigor):

one constraint methodology, with margin, coverage, evidence, review, and fault-tolerance scaled to consequence — safety-critical maximizes provable constraint completeness/correctness across a harsh envelope with documented evidence, while consumer optimizes the same discipline against cost and schedule. The mechanics are identical; the bar for margin, coverage, and especially *demonstrable* correctness rises with the cost of failure.

Follow-up questions: 1. What constraint-related evidence is distinctive to safety-critical certification? 2. Why do exceptions face stricter scrutiny in a safety-critical design?

Follow-up answers: 1. Traceable documentation *proving* constraint completeness and correctness — records that every clock is defined and every register clocked, every exception is justified and valid, and every required mode×corner is covered, with audit trails — because certification requires demonstrable evidence of correctness, not just a clean run; consumer designs rely on the clean run and lighter documentation. 2. Because an exception silently suppresses checking, so an unjustified or invalid one can hide a real violation — and in a safety-critical design a hidden timing failure can have safety consequences, so each exception must be rigorously justified, reviewed, and documented; the tolerance for an unproven exception is essentially zero where safety depends on it.

Q6. How would you defend investment in rigorous constraint methodology and verification to a skeptical manager?

Why the interviewer asks: The complacency-defense question.

Expected answer: I'd frame constraints as the specification the entire timing signoff rests on, and rigorous methodology/verification as the cheapest insurance against the most common and most silent class of timing failure. Points: **(1)** Constraints are the largest source of real timing bugs — not tool inaccuracy, but the tool faithfully executing wrong/incomplete constraints — so investment here targets the highest-probability failure mode. **(2)** The dangerous constraint errors are *silent* (missing clocks leave paths unchecked, wrong exceptions hide violations, coverage gaps produce no symptom) — so without disciplined methodology and verification, they slip to silicon undetected, exactly the escapes that cause respins. **(3)** "Our constraints have always been fine" describes past designs *with* the current rigor — the clean record is the *return* on the methodology, not evidence it's unneeded; and new designs have more clocks, modes, and tighter margins, stressing constraints more. **(4)** A constraint escape (a wrongly-expected real path, an optimistic period) is a respin or field failure — enormously expensive — while the methodology cost (generation, automated checks, review) is modest and mostly one-time. **(5)** Verification (`check_timing`, exception audits, consistency checks) directly catches the silent errors that would otherwise escape. The asymmetry argument: the methodology cost is small and recurring-cheap; a single constraint escape is catastrophic; and the failures are silent so they *will* escape without the rigor. I'd frame the clean record as *evidence the rigor works*, not evidence it's optional — cutting it removes what has been silently preventing the escapes. Rigorous constraint methodology is the cheapest protection against the most probable, most silent, and most expensive class of timing failure.

Follow-up questions: 1. Why is the clean constraint record not evidence that the rigor is unnecessary? 2. What makes constraint escapes especially likely to reach silicon without verification?

Follow-up answers: 1. Because the clean record was *produced* by the rigor being questioned — it's the return on that investment, not proof it's superfluous; removing the methodology removes what has been silently preventing constraint escapes, and new designs (more clocks, modes, tighter margins) stress constraints more than the past ones that built the record. 2. Because constraint errors are *silent* — a missing clock, unconstrained path, or wrong exception produces *no violation* (the path isn't checked or is wrongly excepted), so nothing flags the problem; without active verification (`check_timing`, exception audits, consistency checks) that specifically hunts these silent gaps, they pass through signoff undetected and reach silicon.

Q7. How would you determine which constraint checks to automate/gate versus keep as expert review?

Why the interviewer asks: Automation-vs-judgment for constraint QA.

Expected answer: Automate the mechanical, coverage-oriented, rule-checkable; reserve experts for intent and appropriateness. **Automate/gate:** coverage (`check_timing` — no unconstrained endpoints, missing clocks, missing I/O); relationship completeness (every clock pair related/async/exclusive); period/waveform sanity against a spec table; exception *existence of justification* (every exception has a required comment/entry); consistency (block/top boundary match, same SDC across stages, cross-tool application); design-rule presence. These are deterministic, run as gates on every block/stage, and catch the silent coverage/consistency errors. **Expert review:** whether a declared relationship or exception is *correct* (does this false path reflect real functional intent? is this really async? — a judgment about design intent a checker can't make); whether margins are *right-sized* (justified vs padded); whether the constraint *architecture* is sound; and novel/exotic situations (unusual clocking, complex modes) needing judgment. So automate *completeness, consistency, and rule-conformance* (guaranteeing nothing is silently missed or inconsistent, and every exception at least has a justification), and reserve experts for *correctness of intent and appropriateness of margin/architecture* — the things requiring understanding of what the design is supposed to do. The dividing line is whether a check can be decided from the constraints/netlist alone (automate) or requires knowing design intent (expert). This maximizes coverage of the silent mechanical errors while focusing scarce expert time on the intent-and-judgment questions — especially exception *validity*, which automation can flag for review (missing justification) but only an expert can truly assess (is the justification correct?).

Follow-up questions: 1. Which is automatable: that an exception has a justification, or that the justification is correct? 2. What's the dividing line between automatable and expert-review checks?

Follow-up answers: 1. That an exception *has* a justification is automatable (check every exception has a required comment/entry — a mechanical presence check); whether the

justification is *correct* (does it reflect real functional intent?) requires expert judgment about design intent, which a checker can't determine from the constraints alone. 2. Whether the check can be decided from the constraints and netlist alone (automate — coverage, consistency, rule-conformance, justification presence) versus requiring knowledge of *design intent* (expert review — whether a relationship/exception is correct, whether margin is right-sized, whether the architecture is sound); intent-dependent judgments can't be mechanized.

Q8. How would you assess whether accumulated constraint technical debt threatens a next-generation design?

Why the interviewer asks: Strategic technical-debt assessment.

Expected answer: I'd compare what the next-gen design *demands* of the constraint methodology against its current *capability*, scoping each gap to the specific new pressures. Next-gen pressures: more clocks/domains, higher frequencies (tighter margins), more modes (DVFS, power states), more reuse/hierarchy, possibly new constructs (multi-die, new interfaces). Debt to assess: **(1)** Is SDC still hand-written per block (inconsistency risk that worsens with scale) rather than generated from a spec? **(2)** Are exceptions accumulated ad hoc without justification/audit (stale exceptions that grow dangerous as the design changes)? **(3)** Is there automated coverage/consistency verification, or is correctness assumed (tighter next-gen margins leave no room for silent errors)? **(4)** Are margins principled and correlated, or carried-over magic numbers that may not suit the new node/frequency? **(5)** Can the methodology handle the next-gen's new modes/hierarchy/constructs? For each gap, I'd assess the risk *relative to next-gen demands* — a hand-written-SDC practice tolerable at current scale becomes critical when domain count and margin pressure rise; an unaudited exception pile becomes dangerous as the design changes around stale exceptions. Then prioritize closing the debt the next-gen makes critical — e.g. moving to generated, spec-driven constraints before the domain/mode count explodes, building coverage/exception-audit automation before margins tighten, and re-deriving margins with correlation for the new node. The principle: constraint debt (hand-written SDC, unjustified/stale exceptions, unverified coverage, magic-number margins) is tolerable at current scale but can become critical when next-gen scale, frequency, and complexity exceed what an informal methodology safely handles — so the assessment is forward-looking against specific next-gen demands, closing the newly-critical gaps proactively rather than discovering them mid-project.

Follow-up questions: 1. What next-gen pressures most expose constraint-methodology debt? 2. Why do stale exceptions become more dangerous as a design evolves?

Follow-up answers: 1. More clocks/domains and modes (more relationships and scenarios to get right), higher frequencies and tighter margins (no room for silent errors, more precise constraints needed), more reuse/hierarchy (more boundary-consistency burden), and new constructs (multi-die, new interfaces) — each stressing an informal, hand-written, unverified methodology beyond what it can safely handle. 2. Because a design change can invalidate a previously-valid exception — a path it excepts may become a real functional

path, or its justifying condition may no longer hold — so a stale exception keeps suppressing checking on a path that now needs it; as the design evolves around an unaudited exception pile, more exceptions silently become wrong, each a hidden potential escape.

Q9. A team proposes loosening a constraint (period or margin) to close timing, arguing the real requirement is softer. How do you respond?

Why the interviewer asks: Correcting/evaluating a constraint-loosening rationalization.

Expected answer: I'd evaluate the claim rigorously rather than accept or reject it reflexively. Loosening a constraint to close timing is legitimate *only if* the real requirement genuinely is softer — not just because it makes the numbers work. **(1) Demand the evidence** — what establishes that the real requirement is softer? A period loosening must be justified by the actual target frequency (a spec fact), not by the desire to close; a margin reduction must be justified by the real uncertainty being smaller (correlated data), not by convenience. **(2) Distinguish real from convenient** — is this correcting an over-constraint (the constraint was tighter than the true requirement, so loosening restores accuracy — legitimate), or is it relaxing a real requirement to avoid the work of meeting it (dangerous)? **(3) Check the source of the requirement** — the period comes from the target frequency (a spec/product decision, not an engineering convenience); loosening it changes the product, which is a spec decision, not a timing-closure tactic. Margin comes from real uncertainties; reducing it requires showing the uncertainty is genuinely smaller. **(4) Consider the consequence** — if the loosening is wrong (the requirement really was that tight), the design fails in the field. So I'd support loosening only if it's shown to *correct a genuine over-constraint* (the true requirement is demonstrably softer — a spec fact or correlated margin), with the change owned by whoever owns the requirement (a period/frequency change is a product decision). I'd refuse a loosening that's merely a *convenience to close* — relaxing a real requirement because meeting it is hard is how a design ships that doesn't actually work. The discipline: constraints encode real requirements, so loosening one must be justified by the requirement genuinely being softer (evidence), not by the difficulty of meeting it — and requirement changes belong to the requirement owner, not the timing closure.

Follow-up questions: 1. What distinguishes a legitimate constraint loosening from a dangerous one? 2. Why is loosening the clock period not purely a timing-closure decision?

Follow-up answers: 1. Legitimate: it *corrects a genuine over-constraint* — the constraint was tighter than the true requirement, and evidence (a spec fact, or correlated data showing the real uncertainty is smaller) shows the real requirement is softer. Dangerous: it *relaxes a real requirement* merely because meeting it is hard — a convenience to close that ships a design which doesn't actually meet its real requirement. 2. Because the period comes from the target operating frequency, which is a product/spec decision — loosening it lowers the frequency the product runs at, changing the product's capability; that's a

decision for whoever owns the frequency spec, not a tactic the timing-closure team can adopt unilaterally to make paths close.

Q10. How would you structure knowledge transfer for constraint expertise across a team?

Why the interviewer asks: Organizational resilience for specialized constraint knowledge.

Expected answer: Constraint expertise blends SDC mechanics, timing understanding, and design-intent judgment, so transfer must build all three, anchored in the *why* and real experience. **Document the reasoning and the spec** — the constraint architecture and *why* it's structured that way (which clocks/relationships, why these exceptions and margins), tied to the single source of truth and to real past issues (a constraint bug that caused a respin teaches more than any manual). **Encode mechanical knowledge** in generated-constraint templates and automated checks (durable, self-documenting, and they enforce the practice). **Apprenticeship on real judgment** — have engineers write real constraints, justify exceptions (defending each), size margins (with justification), and *audit* others' SDC under mentorship, since the intent judgments (is this really false? is this margin right?) are learned by doing with feedback. **Teach the silent-failure mindset** — why constraint errors are silent and consequential, so the culture actively verifies coverage and exception validity rather than trusting clean reports. **Preserve the constraint spec, exception justifications, and correlation history** as organizational assets. The goal: multiple engineers can architect constraints, write and justify exceptions correctly, size margins defensibly, and audit SDC — understanding the mechanics, the timing, and the design intent, not just running templates. The deepest constraint knowledge is *judgment about intent* (which paths are truly false/multicycle, what margin is justified) grounded in the silent-failure awareness — so transfer builds that through documented reasoning, real-work apprenticeship, and a verification-minded culture, making the team's constraint capability resilient to any individual's departure. This matters because constraint errors are silent and consequential, so the capability to get constraints right must be distributed, not resting on one expert.

Follow-up questions: 1. Why must constraint knowledge transfer include design-intent judgment, not just SDC syntax? 2. What cultural mindset should accompany constraint expertise?

Follow-up answers: 1. Because the highest-value, highest-risk constraint decisions are judgments about *intent* — whether a path is truly false or multicycle, whether clocks are really async, how much margin is justified — which depend on understanding what the design is supposed to do; teaching only syntax leaves engineers able to write commands but not to make the correctness judgments (especially about exceptions) where the real risk lives. 2. A silent-failure, verification-minded culture — understanding that constraint errors are silent (missing clocks, wrong exceptions, coverage gaps produce no violation), so the team actively verifies coverage, exception validity, and consistency rather than

trusting clean timing reports, treating constraint correctness as something to prove, not assume.

Q11. Ten years out, how do you expect SDC and constraint methodology to evolve, and what stays constant?

Why the interviewer asks: Forward-looking capstone.

Expected answer: Likely evolution: **more generation and abstraction** — constraints increasingly generated from higher-level intent specs (or IP-XACT-like descriptions) rather than hand-written, with tooling that derives SDC from architectural intent; **more automation and ML assistance** — automated exception inference/validation, coverage analysis, and consistency checking, possibly ML-assisted flagging of suspect constraints; **more modes and scenarios** — DVFS, power states, and adaptive operation multiplying the scenarios constraints must express; **multi-die/chiplet** constraints becoming central as designs are assembled from chiplets; **tighter integration** across the flow (unified constraint representation shared by synthesis, P&R, signoff, reducing cross-stage inconsistency); and possibly **new standard formats** evolving beyond today's SDC. **What stays constant:** the fundamentals — a design's timing intent must be *specified* to the tools (clocks, I/O, exceptions, environment); the tool does *exactly* what the constraints say; the dangerous errors are *silent* (missing/wrong constraints produce no violation); and constraint correctness, completeness, and consistency — verified, not assumed — remain the foundation of trustworthy timing. So constraints will get more generated, automated, abstracted, and distributed (multi-die), but the core discipline — specify the timing intent correctly and completely, justify every exception, keep it consistent across stages, and *verify* rather than trust — endures and matters *more* as complexity grows, because more scenarios, more reuse, and tighter margins mean more ways for a silent constraint error to escape. The permanent anchor: constraints are the specification the whole timing signoff rests on, so however the tooling evolves, getting the timing intent specified correctly and completely — and verifying it — remains the foundational discipline, just expressed through progressively more powerful generation and verification tooling.

Follow-up questions: 1. What constraint fundamentals stay constant regardless of tooling evolution? 2. Why will the silent-failure nature of constraint errors persist?

Follow-up answers: 1. That timing intent must be specified to the tools (clocks, I/O, exceptions, environment); that the tool does exactly what the constraints say (no notion of intent); that the dangerous errors are silent; and that constraint correctness, completeness, and consistency must be verified, not assumed — these hold regardless of how the formats and tooling advance. 2. Because it's inherent to how constraints work — the tool checks only what it's told to check, so a missing or wrong constraint means a path is unchecked or wrongly checked, producing *no* violation to flag the error; no matter how tooling evolves, an absent or incorrect specification of intent will always manifest as absent or wrong checking, which is silent by nature — so verification will always be needed to catch it.

Q12. How would you resolve a dispute between two teams over a shared interface's timing budget?

Why the interviewer asks: Cross-team budget conflict resolution.

Expected answer: A shared interface's period must be split between the two sides (each side's clock-to-out, the interconnect, the other's setup, and margins), and disputes over the split are resolved by returning to physical reality and joint optimization. **(1) Establish the fixed facts** — the interface period (from the spec), the interconnect delay (from the physical/board reality), and each side's achievable clock-to-out and setup (from their implementations); much of the budget is *determined*, not negotiable. **(2) Frame it as a shared goal** — both teams need the interface to work; it's a joint optimization, not a zero-sum negotiation, so the aim is a split that lets *both* sides close, allocating the flexible margin where it's cheapest to absorb. **(3) Allocate the genuinely-flexible portion** (the margin, or slack in the split) based on where it's most affordable — the side with more slack absorbs more, minimizing total difficulty. **(4) Use a single source** — the interface budget specified centrally (an interface contract), with both sides' I/O constraints derived from it consistently, so they agree by construction. **(5) If truly contested**, the arbiter is the physical reality (measured/estimated interconnect and achievable device timing) plus the shared objective (both must close). The resolution: most of the budget is a physical fact (interconnect, achievable timing), so the dispute usually dissolves by establishing those facts; the flexible margin is allocated by joint optimization (cheapest side absorbs it) toward the shared goal of both closing, specified centrally so both sides' constraints agree. It's not a territorial negotiation but a jointly-optimized split grounded in physical reality and a shared objective — the interface budget is a *contract* both sides derive from consistently, not a prize to win.

Follow-up questions: 1. Why is much of an interface budget not actually negotiable? 2. How should the genuinely-flexible margin be allocated?

Follow-up answers: 1. Because much of it is determined by physical facts — the interface period (spec), the interconnect delay (board/physical reality), and each side's achievable clock-to-out and setup (their implementations) — these are measured or fixed, not preferences, so they constrain the split objectively rather than being open to negotiation. 2. By joint optimization toward the shared goal of both sides closing — allocate the flexible margin to where it's *cheapest to absorb* (the side with more slack takes more), minimizing total difficulty; it's a cooperative minimization, not a zero-sum split, since both teams need the interface to work.

Q13. How would constraint considerations inform an early architectural or floorplan decision?

Why the interviewer asks: Upstream influence of constraint expertise.

Expected answer: Constraint/timing expertise applied early can steer architecture and floorplan toward timing-friendly choices, avoiding expensive late problems. **Clock-domain**

architecture — advocating fewer, well-related domains (synchronous, timable) over many async domains (each needing synchronizers and CDC), because the constraint/verification cost of async crossings is real; this shapes how many clocks and domains the architecture defines. **Interface choices** — flagging that a high-speed interface needs source-synchronous clocking (a constraint/timing consequence) informs the interface architecture. **Floorplan for clocking** — advising that clock domains be kept physically localized (for low-skew trees), cross-domain interfaces be co-located (compact CDC), and high-frequency domains be placed for tight trees — because the floorplan determines achievable clock skew/latency, which the constraints must then live with. **Budgeting feasibility** — assessing early whether the intended interface/hierarchy budgets are achievable, so the architecture doesn't commit to an infeasible timing split. **Mode complexity** — flagging that many modes/DVFS points multiply the constraint scenarios and verification burden, informing how much mode complexity to build in. The principle: constraint/timing consequences (async-crossing cost, clock-tree feasibility, interface-budget achievability, mode-scenario burden) should inform architecture and floorplan *early*, when they're cheap to influence, rather than being discovered as constraint/closure problems late. I'd translate architectural choices into their constraint/timing cost — "this many async domains means this much CDC and verification; this floorplan enables (or prevents) a low-skew tree; these budgets are (in)feasible" — so architects and floorplanners see the downstream timing consequences of their decisions. Applying constraint expertise upstream prevents the design from being architected in a way that makes timing constraints hard or impossible to satisfy later.

Follow-up questions: 1. How does clock-domain count translate into a constraint/verification cost? 2. Why should floorplanning be clocking-aware from a constraint standpoint?

Follow-up answers: 1. Each independent (async) domain adds crossings that need synchronizers and separate CDC verification, plus relationship declarations and analysis scenarios — so more async domains mean more CDC design/verification burden and more constraint complexity; fewer, well-related (synchronous) domains are directly timable and cheaper, so the domain count maps to a quantifiable constraint/verification cost. 2. Because the floorplan determines the achievable clock-tree skew and latency (localized domains enable low-skew trees; scattered ones force high-skew, high-latency trees), and the constraints must then live with whatever skew the floorplan permits; a clocking-aware floorplan (localized domains, co-located CDC, tight trees for fast domains) enables constraints that can be met, while a clocking-blind one can force skew that makes timing hard to close.

Q14. How would you mentor an engineer from writing basic SDC commands toward true constraint mastery?

Why the interviewer asks: Developing deep constraint capability in others.

Expected answer: Move them from *writing commands* to *reasoning about the timing specification*. **Start with the "why"** — not just `create_clock` / `set_input_delay` syntax, but

what constraints *are* (the specification of timing intent the tool executes literally) and the four categories as a mental model, so they see constraints as a specification, not clerical setup. **Build the path model** — timing paths, what clocks/I-O/exceptions each needs, so they understand *what* each constraint is characterizing. **Teach exceptions as the crux and the danger** — how false/multicycle/max-min work, *why* they're needed, and *why* they're dangerous (silent suppression of checking), so they treat exceptions with the caution they demand. **Instill the silent-failure mindset** — why missing clocks, coverage gaps, and wrong exceptions produce no violation, so they verify (`check_timing`, exception audits) rather than trust clean reports. **Progress to architecture and audit** — have them constrain a real design across all four categories, justify every exception, size margins, and *audit* someone else's SDC — the judgment that defines mastery. **Ground in consequences** — real constraint bugs and silicon escapes, so the stakes are concrete. The goal: they can architect a constraint strategy, write and *justify* exceptions correctly, size margins defensibly, verify coverage, and audit SDC — understanding mechanics, timing, and design intent. Mastery is *reasoning about constraints as the timing specification* — completeness, correctness, exception validity, consistency — grounded in the silent-failure awareness and design intent, not fluency with commands. The mentor's key moves: always teach the *why* behind each constraint, treat exceptions with special rigor (the highest-risk skill), and build the verification mindset that constraints must be proven correct, not assumed — because that mindset, more than any command knowledge, is what makes a trustworthy constraint engineer.

Follow-up questions: 1. What's the core mindset shift from writing SDC to constraint mastery? 2. Why is exception-writing treated as the highest-risk skill to mentor carefully?

Follow-up answers: 1. Seeing constraints as *the timing specification the tool executes literally* — to be made complete, correct, and consistent, with every exception justified and everything verified — rather than as commands to write; mastery is architecting and auditing the specification with judgment about design intent, grounded in the silent-failure awareness, not producing syntactically valid SDC. 2. Because exceptions silently suppress checking, so a wrong, over-broad, or unjustified exception hides real violations with no symptom — the highest-consequence, most-silent constraint error; mentoring it carefully (why each is needed, why dangerous, how to justify and scope) builds the caution and judgment that prevent the most damaging class of constraint mistake.

Q15. How should an organization weigh dependence on constraint-generation and verification tools/flows?

Why the interviewer asks: Tool-dependence for the constraint methodology.

Expected answer: A mature methodology depends on tools/flows for generation (deriving SDC from a spec) and verification (`check_timing`, consistency and exception checks), and this dependence is beneficial but must be managed. Benefits: generation eliminates hand-writing errors and enforces consistency; automated verification catches silent coverage/consistency errors at scale — both hard to achieve manually. Risks: **(1)** A bug in the *generator* propagates a systematic error to all generated SDC (a wrong template affects

everything) — so generators must be validated and version-controlled like critical code. **(2)** Over-trust in *verification* tools — if a check has a gap (doesn't catch a certain error class), the org may assume coverage it doesn't have; so the checks' completeness must be understood, not blindly trusted. **(3)** Loss of underlying understanding — if engineers only run the flow, they may lose the ability to judge its output or handle cases it doesn't cover. Managing it: validate and version-control generators (they're critical code); understand what each verification check does and doesn't cover (no false confidence); retain engineers who understand the underlying constraint principles (to judge outputs and handle exceptions); and keep the methodology auditable. So the dependence is embraced (it's how correctness and consistency scale) but *managed* — generators validated as critical code, verification coverage understood rather than blindly trusted, and underlying expertise retained — rather than becoming blind reliance. The principle mirrors other tool dependence: unavoidable and valuable at scale, so managed through validation, understanding of coverage, and retained expertise — with the specific caution that a *generator* bug or a *verification* gap has broad, systematic reach, so both deserve the scrutiny given to critical infrastructure, not the trust given to a black box.

Follow-up questions: 1. Why does a constraint-*generator* bug deserve special scrutiny? 2. What's the risk of blindly trusting the verification checks?

Follow-up answers: 1. Because the generator produces *all* the SDC from the spec, so a bug in it (a wrong template, a mis-derived relationship) propagates a systematic error to every block/stage at once — a broad, systematic reach — so generators must be validated and version-controlled like critical code, not assumed correct. 2. If a verification check has a gap (doesn't catch a certain error class), blindly trusting it creates false confidence — the org assumes a coverage it doesn't actually have, so errors in the uncovered class slip through believed-verified; the checks' completeness must be understood so the org knows what is and isn't actually being caught.

Q16. What would justify signing off a design whose closure depends heavily on aggressive (tight) constraints or exceptions?

Why the interviewer asks: Judgment about signing off on aggressive constraint choices.

Expected answer: If closure depends on aggressive constraints (tight margins) or on exceptions (paths excepted to avoid violations), the design's correctness *rests on those choices being valid* — so I'd sign off only if each is rigorously justified, not merely convenient. For **tight margins/constraints:** the tightness must be *validated* (correlated against worst-case silicon/characterization showing the real requirement genuinely is that tight and the margin genuinely sufficient), not chosen to make numbers close; and the design shouldn't be on a knife-edge where a small error flips it to failure. For **exceptions that enable closure:** every exception the closure depends on must be *confirmed valid* — the excepted paths must genuinely be false or multicycle by real functional intent (verified with the designer/spec), not excepted to silence violations; an over-broad or invalid exception "closing" the design is a hidden failure. I'd specifically scrutinize: is closure achieved by *correctly* excepting genuinely-non-real paths (legitimate) or by *wrongly*

excepting real paths (dangerous optimism disguised as closure)? Is the tight margin *validated* (real requirement, correlated) or *convenient* (chosen to close)? I'd refuse to sign off if closure depends on unvalidated tight margins or unconfirmed exceptions — because then the "closure" may be fictional (real paths hidden by bad exceptions, or real margin absent). I'd sign off only when every aggressive choice the closure depends on is independently justified — exceptions confirmed to reflect real functional intent, tight margins validated against real worst-case data — so the closure reflects a design that genuinely meets its real requirements, not one whose passing report rests on optimistic constraints or invalid exceptions. The deeper the closure's dependence on aggressive choices, the higher the bar for validating each, because the design's real-world correctness hinges on them being right.

Follow-up questions: 1. How do you distinguish legitimate from dangerous exception-driven closure? 2. Why does heavier dependence on aggressive choices raise the signoff bar?

Follow-up answers: 1. Legitimate: closure is achieved by *correctly* excepting paths that are genuinely false or multicycle by real functional intent (confirmed with designer/spec) — the excepted paths truly don't need timing. Dangerous: closure is achieved by *wrongly* excepting real paths to silence violations — an invalid or over-broad exception hides a genuine timing problem, so the passing report is fictional; the distinction is whether each exception reflects confirmed real intent or just makes the violation disappear. 2. Because the design's real-world correctness *rests on* those aggressive choices being valid — if a tight margin or an exception the closure depends on is wrong, the design fails in silicon despite the clean report; the more the closure hinges on such choices, the more of the design's correctness depends on each being right, so each must be validated more rigorously to justify trusting the signoff.

Q17. How would you weigh adopting a new constraint methodology or format against a proven flow?

Why the interviewer asks: Technology-adoption judgment for methodology.

Expected answer: Separate the new approach's benefit from its adoption risk, weighing the proven flow's validated reliability. A new constraint methodology/format (a new generation framework, a new standard, an ML-assisted tool) may offer real benefits — better consistency, less manual effort, new capability — but the proven flow carries validated correctness and team familiarity. Evaluation: **(1) Benefit magnitude** — does it solve a real problem the proven flow can't (e.g. handling a new construct, eliminating a class of manual error)? **(2) Maturity and validation** — is the new methodology proven to produce *correct* constraints? A methodology that generates SDC must be validated as thoroughly as the constraints it produces, because a flaw propagates systematically; unproven generation is a systematic risk. **(3) Verification compatibility** — do the existing verification checks still work, or does the new approach need new validation? **(4) Migration risk** — can it be adopted incrementally (on a non-critical block first) to build confidence, with fallback to the proven flow? **(5) Team readiness** — does the team

understand it well enough to judge its output? The principle (adoption judgment): a new constraint methodology must *earn* trust through validation — proven to produce correct, verifiable constraints — before replacing a proven flow, because constraints are the timing specification and a systematic methodology flaw has broad reach. So adoption favors: a clear benefit the proven flow lacks, demonstrated correctness (validated generation/verification), incremental migration with fallback, and team readiness — capturing genuine methodology advances while protecting the validated reliability of the proven flow. I'd pilot the new approach on a non-critical block, validate its output against the proven flow, and adopt broadly only once its correctness is demonstrated — never betting a critical design on an unvalidated new constraint methodology, given the systematic blast radius of a methodology flaw.

Follow-up questions: 1. Why must a new constraint-generation methodology be validated especially thoroughly? 2. How would you de-risk adopting a new methodology?

Follow-up answers: 1. Because it generates the constraints — the timing specification the whole signoff rests on — so a flaw in the methodology propagates a systematic error to all the SDC it produces (broad blast radius), unlike a localized bug; it must be validated as thoroughly as the constraints themselves before being trusted, since an unproven generation methodology is a systematic risk. 2. Adopt it incrementally — pilot on a non-critical block first, validate its generated constraints against the proven flow (confirming correctness), keep the proven flow as fallback, and expand only once correctness is demonstrated and the team understands it well enough to judge its output — capturing the benefit while limiting exposure to any methodology flaw.

Q18. When would you sign off on removing a long-standing constraint or margin carried across generations?

Why the interviewer asks: The margin/constraint-removal governance question.

Expected answer: Never remove it just because its origin is forgotten — an unexplained long-standing constraint or margin is *presumed load-bearing*. Such constraints/margins often encode hard-won lessons (a past escape, a silicon surprise, a correlation gap), so removing one blindly risks re-exposing the exact problem it was added to prevent. I'd reconstruct what it protects: trace its history (was it added after a specific issue or silicon finding?), consult documentation/margin-ledger and the people who carried it, and determine the real effect it covers. Then assess whether that effect is *now* handled otherwise: if it covered an uncertainty now computed elsewhere (e.g. a margin now covered by a real analysis), or a condition no longer present, it may be safe to remove; if it covers a real, still-present effect (a worst-case uncertainty, a known model optimism, a correlation gap) not otherwise captured, removing it re-exposes that risk. Validate rigorously with *worst-case* correlation — does the design still meet its real requirements without it, across worst-case conditions? Quantify the exposure. I'd sign off removal *only* when I understand precisely what it covered, can show that effect is now non-existent or covered elsewhere, and have validated (against worst-case data) that the design remains correct without it — never merely because closure or performance wants the margin back,

and never because "we don't know why it's there so it's probably unneeded" (the opposite inference is safer: unknown origin means presumed-important until proven otherwise). The culminating principle: constraints and margins are presumed to protect against real, often silent risks until *proven* otherwise through reconstructed understanding and worst-case validation — the burden of proof is on removal, because the downside of wrongly removing a load-bearing margin (a re-exposed escape) vastly outweighs the upside (a little recovered performance).

Follow-up questions: 1. Why is an unexplained long-standing margin presumed load-bearing? 2. What must be established before removing such a margin?

Follow-up answers: 1. Because such margins often encode hard-won lessons — added after a real escape, silicon surprise, or correlation gap — so an unknown origin more likely means the reason is forgotten than that there was none; the safer inference is that it protects against a real (possibly silent) risk until proven otherwise, since wrongly removing a load-bearing margin re-exposes the problem it was preventing. 2. A reconstructed understanding of precisely what effect it covers (from history, documentation, and people), proof that the effect is now non-existent or covered elsewhere, and worst-case validation (correlated data) that the design still meets its real requirements without it — the burden of proof is on removal, so all three must be established before signing off, not just a desire to recover the margin.

Q19. How do you balance the completeness of constraint verification against schedule and effort?

Why the interviewer asks: The practical tension.

Expected answer: Completeness of *real* constraint coverage is non-negotiable (constraint errors are silent and reach silicon), but achieved *efficiently* through automation, prioritization, and reuse — not brute force. **(1) Automate the coverage/consistency/justification checks** — `check_timing`, relationship completeness, exception-justification presence, block/top and cross-stage consistency — so they run cheaply as gates on every block/stage, guaranteeing the silent errors are always caught without manual effort. **(2) Prioritize expert review by risk** — focus scarce expert time on the highest-risk areas: exception *validity* (the top risk), critical clock domains, and complex modes — where automation can flag but only judgment can confirm. **(3) Reuse verified constraints** — generated-from-spec SDC and validated templates mean much verification is inherited, not redone per block. **(4) Incremental verification** — after an ECO or change, re-verify what changed, not everything. **(5) Never cut coverage** — the completeness of checking that nothing is unconstrained or wrongly excepted — because that's where the silent escapes are. The principle: never sacrifice *coverage* of the silent errors (missing constraints, invalid exceptions, inconsistencies) to save schedule — that's how escapes reach silicon; but achieve that coverage *efficiently* — automation makes complete coverage/consistency/justification checking cheap, prioritization focuses expert judgment on exception validity and high-risk areas, and generation/templates make verification inheritable. The judgment is distinguishing *coverage* (never cut — the silent-error protection) from *redundant manual*

effort (cut via automation, reuse, and incremental verification). For constraints especially, because the errors are silent and reach silicon, the bar for complete coverage is high, so efficiency must come from automating and prioritizing the verification, never from skipping it — a skipped check on a silent-error class is an invitation for an escape.

Follow-up questions: 1. What constraint verification must never be cut for schedule, and why? 2. How is complete coverage made affordable?

Follow-up answers: 1. Coverage of the silent errors — that nothing is left unconstrained (missing clocks, unconstrained endpoints), no exception is invalid/over-broad (wrongly suppressing real paths), and constraints are consistent across blocks/stages — because these errors produce no violation and reach silicon undetected, so skipping their verification to save schedule directly invites catastrophic escapes. 2. Through automation (coverage, consistency, and justification-presence checks run cheaply as gates on every block/stage), prioritization (scarce expert review focused on the highest-risk judgments, especially exception validity), and reuse (generated-from-spec constraints and validated templates make verification inheritable) — plus incremental re-verification of only what changed — making complete coverage affordable rather than achieving speed by skipping it.

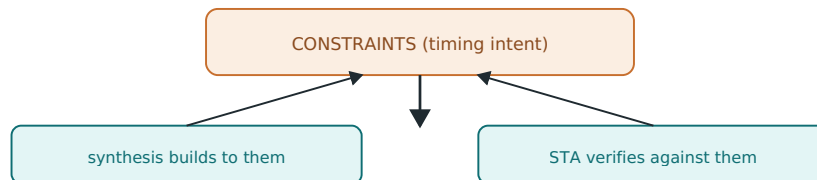
Q20. Synthesize: what is the essence of SDC and constraints, and why are they the foundation of timing signoff?

Why the interviewer asks: The integrative capstone for the chapter.

Expected answer: The essence of SDC and constraints is **the specification of a design's timing intent and environment — the clocks, the boundary (I/O) timing, the exceptions, and the operating conditions — communicated to the tools in a form they execute literally, so that synthesis can build a netlist to meet it and static timing analysis can verify the netlist against it.** Constraints are the foundation of timing signoff because *the entire signoff is only as meaningful as the constraints*: STA doesn't know design intent — it does exactly what the constraints say, checking only what they define, against the budgets they specify. So the constraints *are* the specification of correct timing behavior; if they're right, the signoff verifies the real design against its real requirements, and if they're wrong, the signoff is a confident verification of the wrong thing. The distinctive danger is that constraint errors are *silent*: a missing clock leaves paths unchecked, a wrong exception hides real violations, a coverage gap or an optimistic constraint produces no symptom — the report looks clean while real timing goes unverified or wrongly verified. And because the same constraints drive synthesis, place-and-route, and signoff, they must mean the same thing at every stage, or the design passes one and fails another. This is why constraints are written first (everything depends on the clocks), reviewed most carefully (especially exceptions, which silently suppress checking), verified for completeness and consistency (not trusted because the tool ran), and treated with the rigor of a specification — because they *are* the specification the whole timing signoff rests on. Mastering constraints — specifying the timing intent correctly and completely across all four categories, justifying every exception, keeping the specification consistent across

stages, and actively verifying rather than assuming — is therefore the foundational discipline of timing signoff, the one on which the trustworthiness of every timing number, and ultimately the design's real-world timing correctness, depends. Correct RTL with wrong constraints still yields a failing chip; the constraints are where timing correctness is truly specified, and getting them right is as fundamental as the logic itself.

Constraints are the specification the signoff rests on



Wrong constraints -> confident verification of the wrong thing -> failing chip from correct RTL.

Follow-up questions: 1. Why is the timing signoff only as meaningful as the constraints? 2. What makes constraint errors uniquely dangerous compared to other timing issues?

Follow-up answers: 1. Because STA has no notion of design intent — it does exactly what the constraints say, checking only the paths they define against the budgets they specify — so the constraints *are* the specification of correct timing behavior; if they're wrong (missing clock, bad exception, optimistic budget), the signoff confidently verifies the wrong requirements, meaning a clean signoff guarantees only that the design meets whatever the constraints said, which is worthless if they're wrong. 2. They're *silent* — a missing clock, coverage gap, wrong exception, or optimistic constraint produces *no* violation (the path is unchecked, wrongly excepted, or checked against a wrong budget), so the error doesn't announce itself and a clean report can hide serious unverified or mis-verified timing; combined with driving synthesis/P&R/signoff (so inconsistency makes a design pass one stage and fail another), this silent, foundational nature makes constraint errors both the most consequential and the hardest to catch.

Chapter 1 — Chapter Summary

SDC (Synopsys Design Constraints) is a standard, Tcl-based format for specifying a design's timing intent and environment to synthesis and timing-analysis tools. The netlist describes what the logic *is*; SDC describes how it must *behave in time* — and it does so in a portable form that flows through the entire implementation flow, so the same intent guides synthesis (which builds a netlist to meet it), place-and-route (which optimizes against it), and signoff STA (which verifies against it). Almost every constraint falls into one of four categories: **clocks** (what the clocks are — the foundation everything references), **I/O timing** (what happens at the boundary — input/output delays and the drive/load environment), **timing exceptions** (which paths are special — false paths, multicycle paths, min/max delays), and **environment/design rules** (under what conditions and limits — operating conditions, max transition/capacitance). The unit all of this constrains is the

timing path (from a launch flop or input port, through logic, to a capture flop or output port). Because SDC is Tcl, it is evaluated in order and object queries that match nothing silently constrain nothing — one of many *silent* failure modes. The defining truth of the chapter is that the tool does *exactly* what the constraints say, so constraint errors become timing errors, and the dangerous ones are silent: a missing clock leaves paths unchecked, a wrong exception hides real violations, a coverage gap or optimistic constraint produces no symptom, and an inconsistency between stages makes a design pass one and fail another. Constraints are therefore the specification the entire timing signoff rests on — written first, reviewed most carefully (especially exceptions), verified for completeness and consistency rather than trusted, and treated with the rigor of a specification. Correct RTL with wrong constraints still yields a failing chip; mastering constraints is as foundational to a working design as the logic itself.

Top 20 Quick Revision Questions (Chapter 1)

1. **What is SDC?** A standard Tcl-based format specifying a design's timing intent and environment (clocks, I/O, exceptions, conditions) to synthesis and STA tools.
2. **What does the netlist describe vs SDC?** Netlist = the logic structure (what it is); SDC = the timing intent and environment (how it must behave in time).
3. **What are the four constraint categories?** Clocks, I/O timing, timing exceptions, and environment/design rules.
4. **What is a timing path?** The route from a start point (flop clock pin or input port) through logic to an end point (flop data pin or output port) — the unit STA analyzes.
5. **Why is SDC written in Tcl?** It's executable — commands operate on queried objects, evaluated in order; enables parameterization but a query matching nothing silently does nothing.
6. **What does `create_clock` do, and why first?** Defines a clock's period and waveform at a source; written first because everything (I/O, exceptions, all timing) references clocks.
7. **What does `set_input_delay` model?** How long after the reference clock edge external data arrives at an input port (external clock-to-out + interconnect).
8. **What does `set_output_delay` model?** How long before the external clock edge data must leave an output port (external setup + interconnect).
9. **Max vs min I/O delay?** Max (latest arrival) for setup; min (earliest arrival) for hold — both facets are needed.
10. **What is a false path?** A path that is not a real functional timing path and should not be checked (`set_false_path`).
11. **What is a multicycle path?** A path functionally allowed more than one cycle to propagate (`set_multicycle_path`); its hold multiplier is essential.
12. **What are `set_max_delay` / `set_min_delay` for?** Explicit path delay requirements overriding the default clock-derived timing — e.g. for combinational or async paths.
13. **What do design-rule constraints do?** Limit slew (`set_max_transition`), load (`set_max_capacitance`), and fanout — electrical quality, separate from timing slack.

14. **What is `set_case_analysis`?** Sets a constant on a pin to select a mode/configuration and disable inactive paths, reducing false violations.
15. **What is a virtual clock?** A clock with no source object, used as an external timing reference for I/O delays.
16. **What is the period-frequency relationship?** Frequency = 1/period; the period is the single-cycle setup budget, so it sets the max operating frequency.
17. **"Tool does exactly what constraints say" — meaning?** The tool has no notion of intent; constraint errors become timing errors, and the dangerous ones are silent (no violation).
18. **Why must SDC be consistent across stages?** Synthesis builds to it, P&R optimizes to it, signoff verifies to it; drift makes a design pass one stage and fail another.
19. **What does `check_timing` do?** Reports constraint coverage gaps — unconstrained endpoints, missing clocks, undefined I/O — the silent-error check.
20. **Why are constraints as important as RTL?** They specify the timing requirements STA verifies against; wrong constraints mean a design verified against the wrong thing — a failing chip from correct RTL.

Important Definitions (Chapter 1)

- **SDC (Synopsys Design Constraints):** A standard, Tcl-based format specifying a design's timing intent and environment to synthesis and timing tools.
- **Timing path:** The route from a start point (flop clock pin or input port) through combinational logic to an end point (flop data pin or output port).
- **Start point / end point:** Where a path launches (flop clock pin / input port) and captures (flop data pin / output port).
- **Clock constraint:** A definition of a clock (period, waveform, latency, uncertainty, relationships).
- **Input/output delay:** The timing of data arriving at an input port / required at an output port, relative to a clock edge.
- **Timing exception:** A constraint overriding default single-cycle timing — false path, multicycle path, or min/max delay.
- **False path:** A path declared not a real functional timing path, excluded from checking.
- **Multicycle path:** A path allowed more than one clock cycle to propagate.
- **Design-rule constraint:** A limit on electrical quality (max transition, max capacitance, max fanout), independent of timing slack.
- **Case analysis:** A fixed logic value on a pin selecting a mode/configuration and disabling inactive paths.
- **Virtual clock:** A clock with no source object, used as an external reference for I/O timing.
- **Operating conditions (PVT):** The process, voltage, and temperature the design is analyzed under.

- **check_timing**: A command reporting constraint coverage gaps (unconstrained endpoints, missing clocks, undefined I/O).

Important Commands (Chapter 1)

- `create_clock -name <n> -period <p> [-waveform {r f}] <source>` — define a primary clock.
- `create_clock -name <n> -period <p>` (no source) — define a virtual clock.
- `set_input_delay -max|-min <v> -clock <clk> [get_ports <p>]` — external arrival timing at an input.
- `set_output_delay -max|-min <v> -clock <clk> [get_ports <p>]` — external requirement at an output.
- `set_driving_cell -lib_cell <cell> [get_ports <p>]` / `set_input_transition <v> [get_ports <p>]` — model input drive/edge.
- `set_load <cap> [get_ports <p>]` — model external output load.
- `set_false_path -from ... -through ... -to ...` — exclude a path from timing.
- `set_multicycle_path <N> -setup ... ; set_multicycle_path <N-1> -hold ...` — relax a path to multiple cycles (with hold companion).
- `set_max_delay <v> -from ... -to ...` / `set_min_delay <v> ...` — explicit path delay requirement.
- `set_max_transition <v> ...` / `set_max_capacitance <v> ...` / `set_max_fanout <v> ...` — design-rule limits.
- `set_case_analysis <0|1> [get_pins <p>]` — set a constant to select a mode/disable paths.
- `set_operating_conditions ...` — select the PVT corner.
- `check_timing` — report constraint coverage gaps.
- `report_timing` — detailed path timing breakdown and slack.

Quick Revision Notes (Chapter 1)

- SDC specifies timing intent and environment to the tools; the netlist is what the logic *is*, SDC is how it must *behave in time* — and the same SDC flows through synthesis, P&R, and signoff.
- Four constraint categories: clocks (foundation), I/O timing (boundary), exceptions (special paths), environment/design rules (conditions/limits).
- The timing path (start point → logic → end point) is the unit constrained; clocks budget flop-to-flop paths, I/O delays budget boundary paths, exceptions modify specific paths.
- SDC is Tcl: evaluated in order, programmable, but a query matching nothing silently constrains nothing — one of several silent failure modes.

- The tool does exactly what the constraints say — constraint errors become timing errors, and the dangerous ones are silent (a missing clock or wrong exception produces no violation).
- I/O delays encode the external device timing (clock-to-out, setup) plus interconnect; specify both max (setup) and min (hold) facets.
- Exceptions (false, multicycle, max/min delay) override default timing and are the highest-risk constraints — a wrong or over-broad one silently hides real violations, so each must be justified and narrowly scoped.
- Design-rule constraints (max transition/capacitance/fanout) enforce electrical quality independent of timing slack — a path can meet timing yet violate a design rule.
- Constraints must be consistent across synthesis, P&R, and signoff (single source), or a design passes one stage and fails another; and coverage must be verified (`check_timing`), not assumed from a clean run.
- Constraints are the specification the whole timing signoff rests on — written first, reviewed most carefully (especially exceptions), and verified for completeness/consistency rather than trusted; correct RTL with wrong constraints still yields a failing chip.